

Ronivaldo Domingues de Andrade

Git e GitHub na Prática: do Primeiro Commit à Automação

Rio de Janeiro - RJ
Edição 2025 · Revisão 2026

Ronivaldo Domingues de Andrade

Git e GitHub na Prática: do Primeiro Commit à Automação

Guia prático da capacitação de Git e GitHub, com duração de duas horas, elaborado para os membros da liga acadêmica. Cobre conta e benefícios de estudante, instalação no Windows, no Linux e no macOS, Git, GitHub CLI, Git LFS, commits, merges, Pull Requests, GitHub Pages, GitHub Actions e assinatura de commits.

Rio de Janeiro - RJ
Edição 2025 · Revisão 2026

Lista de ilustrações

Figura 1 - Página de criação de conta no GitHub	12
Figura 2 - Primeira visão do GitHub depois de criar a conta	13
Figura 3 - Adicionando o e-mail acadêmico	15
Figura 4 - Página do Student Developer Pack (education.github.com/pack)	16
Figura 5 - Education benefits nas configurações, com o botão Start an application	16
Figura 6 - Formulário da candidatura	17
Figura 7 - Conferindo o winget	18
Figura 8 - Buscando o Git no winget	19
Figura 9 - Instalação do Git	19
Figura 10 - Instalando o GitHub CLI	20
Figura 11 - Versão do Git	21
Figura 12 - Versão do GitHub CLI	21
Figura 13 - Configurando nome e e-mail	25
Figura 14 - <code>gh auth login:</code> escolhendo o GitHub.com	26
Figura 15 - <code>gh auth login:</code> escolhendo o protocolo	27
Figura 16 - <code>gh auth login:</code> autenticando o Git com as credenciais do GitHub	27
Figura 17 - <code>gh auth login:</code> login pelo navegador	28
Figura 18 - <code>gh auth login:</code> código de uso único	28

Lista de tabelas

Tabela 1 - Roteiro de tempo da capacitação	10
Tabela 2 - Diferenças entre o GitHub Free e o GitHub Pro (conta pessoal)	14
Tabela 3 - As três áreas do Git	24
Tabela 4 - Respostas do <code>gh auth login</code>	26
Tabela 5 - Comandos básicos do Git	30
Tabela 6 - Comandos básicos do GitHub CLI	30
Tabela 7 - Fontes de publicação do GitHub Pages	48
Tabela 8 - Vocabulário do GitHub Actions	49
Tabela 9 - Comandos do Git	71
Tabela 10 - Comandos do GitHub CLI (<code>gh</code>)	76

Lista de abreviaturas e siglas

2FA	<i>Two-Factor Authentication</i> , autenticação em dois fatores
CI/CD	<i>Continuous Integration / Continuous Delivery</i> , integração e entrega contínuas
CLI	<i>Command Line Interface</i> , interface de linha de comando
GCM	<i>Git Credential Manager</i> , gerenciador de credenciais do Git
GPG	<i>GNU Privacy Guard</i> , programa de criptografia e assinatura digital
LFS	<i>Large File Storage</i> , extensão do Git para arquivos grandes
PAT	<i>Personal Access Token</i> , token de acesso pessoal
PR	<i>Pull Request</i> , pedido de integração de uma branch em outra
SSH	<i>Secure Shell</i> , protocolo de acesso seguro por par de chaves
YAML	<i>YAML Ain't Markup Language</i> , formato dos arquivos de workflow

Sumário

Lista de ilustrações	2
Lista de tabelas	3
Sumário	5
1 INTRODUÇÃO	9
1.1 Objetivos de aprendizagem	9
1.2 O roteiro das duas horas	10
1.3 Como ler este guia	10
1.4 Pré-requisitos	11
2 A CONTA NO GITHUB	12
2.1 O que é o GitHub	12
2.2 Criando a conta	12
2.3 Autenticação em dois fatores (2FA)	13
2.4 Benefícios para estudantes	14
2.4.1 O que o GitHub Education oferece	14
2.4.2 Quem pode pedir	14
2.4.3 Passo a passo	15
3 PREPARANDO O AMBIENTE	18
3.1 Windows	18
3.1.1 Conferir o winget	18
3.1.2 Instalar o Git	19
3.1.3 Instalar o GitHub CLI	20
3.1.4 Conferir	20
3.2 Linux (Debian e Ubuntu)	22
3.3 macOS	22
3.4 Conferindo nos três sistemas	23
4 GIT E GITHUB CLI: CONFIGURAÇÃO E LOGIN	24
4.1 O que é o Git	24
4.2 O que é o GitHub CLI	24
4.3 Dizendo ao Git quem é você	24
4.4 Fazendo login no GitHub	26
4.4.1 Pelo GitHub CLI (recomendado)	26

4.4.2	Por chave SSH (alternativa)	29
4.4.3	Por token de acesso pessoal (para scripts)	29
4.5	Comandos do dia a dia	30
5	GIT LFS: ARQUIVOS GRANDES	31
5.1	O problema	31
5.2	A solução	31
5.3	Instalação	31
5.4	Usando	32
5.5	Limites e custos no GitHub	32
6	COMMITTS, MERGES E PULL REQUESTS	34
6.1	Introdução	34
6.2	Commits	34
6.2.1	O que é um commit	34
6.2.2	Boas práticas de commits	34
6.2.3	Fazendo commits — passo a passo	34
6.2.4	Editar o último commit / corrigir mensagens	35
6.2.5	Desfazer / alterar staging	35
6.3	Merges	36
6.3.1	Tipos de merge	36
6.3.2	Merge local com merge commit (passo a passo)	36
6.3.3	Rebase (passo a passo) — para um histórico linear	37
6.3.4	Resolver conflitos — passo a passo	37
6.3.5	Squash e reescrita de commits (passo a passo)	38
6.4	Pull Requests (PR)	38
6.4.1	O que é um Pull Request	38
6.4.2	Fluxo básico — criando um PR (via web)	38
6.4.3	Criar e gerenciar PRs via GitHub CLI (passo a passo)	39
6.4.4	Checklist para revisão de Pull Request	40
6.4.5	Depois do merge — limpeza e sincronização	40
6.5	Boas práticas e recomendações finais	40
6.6	Exemplos rápidos de comandos úteis	41
7	PADRÕES DE COMMIT	42
7.1	Por que padronizar	42
7.2	Conventional Commits	42
7.2.1	Os tipos	43
7.3	Boas práticas para a mensagem	44
7.4	Gitmoji: emojis nas mensagens	45

7.5	Um padrão combinando os dois	46
7.6	Ferramentas	46
8	GITHUB PAGES E GITHUB ACTIONS	48
8.1	GitHub Pages: um site a partir do repositório	48
8.1.1	As duas formas de publicar	48
8.2	GitHub Actions: automação dentro do GitHub	48
8.2.1	Exemplo 1: publicar um site pelo Actions	49
8.2.2	Exemplo 2: rodar um script e guardar o resultado	50
8.2.3	E o [skip ci]?	51
8.2.4	Acompanhando	51
9	ASSINATURA DE COMMITS COM GPG E SSH	52
9.1	O problema: o autor de um commit é só um texto	52
9.2	GPG ou SSH?	53
9.3	Assinando com GPG	53
9.3.1	Instalar o GnuPG	53
9.3.2	Passo 1 — Ver se você já tem uma chave	54
9.3.3	Passo 2 — Gerar a chave	54
9.3.4	Passo 3 — Descobrir o ID da chave	54
9.3.5	Passo 4 — Exportar a chave pública	54
9.3.6	Passo 5 — Cadastrar no GitHub	55
9.3.7	Passo 6 — Dizer ao Git qual chave usar	55
9.3.8	Passo 7 — Ensinar o GPG a pedir a senha no terminal	55
9.3.9	Passo 8 — Testar	55
9.4	Assinando com SSH	56
9.4.1	Requisitos	56
9.4.2	Passo 1 — Ter uma chave SSH	56
9.4.3	Passo 2 — Dizer ao Git para assinar com SSH	56
9.4.4	Passo 3 — Cadastrar como chave de assinatura no GitHub	56
9.4.5	Passo 4 — Permitir a conferência no seu computador	57
9.4.6	Passo 5 — Testar	57
9.5	Conferindo no GitHub	58
9.6	Cuidados com a chave	58
9.7	Problemas comuns	59
10	EXERCÍCIOS PRÁTICOS	60
10.1	Exercício 1 — Conferir as ferramentas	60
10.2	Exercício 2 — O primeiro repositório	60
10.3	Exercício 3 — Uma licença criada pelo site	61

10.4	Exercício 4 — Resolver um conflito	61
10.5	Exercício 5 — Um site no GitHub Pages	63
10.6	Exercício 6 — Automação com GitHub Actions	63
10.7	Exercício 7 — Fork, Pull Request e lista de presença	64
10.8	Exercício 8 — Mensagens de commit	65
10.9	Exercício 9 — Um commit assinado	65
10.10	Checklist	65
11	CONCLUSÃO E PRÓXIMOS PASSOS	67
11.1	Para continuar estudando	67
	REFERÊNCIAS	68
	 APÊNDICES	 70
	APÊNDICE A – COMANDOS DO GIT	71
	APÊNDICE B – COMANDOS DO GITHUB CLI	76
	APÊNDICE C – GLOSSÁRIO	85
	 ANEXOS	 86
	ANEXO A – LISTA DE PRESENÇA	87

1 Introdução

O GitHub é hoje a principal plataforma de desenvolvimento colaborativo. Ele guarda o código, registra quem mudou o quê, organiza a revisão entre colegas e ainda publica sites e roda automações. Tudo isso em volta de uma ferramenta que vem antes dele: o **Git**.

Este guia acompanha a capacitação de Git e GitHub da liga acadêmica. A apresentação dura duas horas; aqui está tudo o que foi dito com mais calma, os passos completos para Windows, Linux e macOS, e os exercícios para praticar depois.

1.1 Objetivos de aprendizagem

Ao final, o participante é capaz de:

1. criar e proteger a conta no GitHub, com autenticação em dois fatores, e pedir os benefícios de estudante;
2. instalar e configurar o Git e o GitHub CLI no próprio sistema;
3. registrar alterações com commits bem escritos e trabalhar com branches;
4. integrar trabalho com merge e resolver um conflito;
5. contribuir com um projeto de outra pessoa por fork e Pull Request;
6. publicar um site com o GitHub Pages e automatizar uma tarefa com o GitHub Actions;
7. versionar arquivos grandes com o Git LFS e assinar commits.

1.2 O roteiro das duas horas

Tabela 1 – Roteiro de tempo da capacitação

Bloco	Conteúdo	Início	Duração
0	Abertura, objetivos e combinados	00:00	5 min
1	A conta no GitHub, 2FA e benefícios de estudante	00:05	15 min
2	Instalação, configuração e autenticação	00:20	20 min
3	Git no dia a dia: commits, branches, merge e conflito	00:40	20 min
—	<i>Pausa</i>	01:00	5 min
4	Colaboração: fork, Pull Request e padrões de commit	01:05	20 min
5	GitHub Pages e GitHub Actions	01:25	15 min
6	Git LFS e assinatura de commits	01:40	15 min
7	Encerramento e próximos passos	01:55	5 min
Total			120 min

1.3 Como ler este guia

Os comandos aparecem em blocos como este. Linhas que começam com # são comentários e não precisam ser digitadas:

```
# confere a versão do Git instalada
git --version
```

Ao longo do texto, três tipos de caixa chamam a atenção:

Conceito

Uma definição importante, para guardar.

Dica

Um atalho ou uma boa prática.

Atenção

Um erro comum, ou um cuidado que evita dor de cabeça.

Quando um comando traz algo entre <sinais>, como <seu-usuario>, troque tudo, inclusive os sinais, pelo valor real.

Qual terminal usar

No **Windows**, use o **Git Bash**, que vem com o Git (capítulo 3). Os comandos deste guia, como `ls`, `touch` e `mkdir -p`, são do **Bash** e não funcionam no Prompt de Comando. No **Linux** e no **macOS**, use o Terminal do sistema. A única exceção é a instalação pelo `winget`, feita no PowerShell.

1.4 Pré-requisitos

- Um computador com Windows 10 (versão 1809 ou mais nova), Windows 11, Linux ou macOS, com permissão para instalar programas.
- Um endereço de e-mail. Para os benefícios de estudante, é melhor ter o e-mail acadêmico.
- Um celular com um aplicativo autenticador (Google Authenticator, Microsoft Authenticator ou similar), para a autenticação em dois fatores.

Não é preciso saber programar. Os exercícios usam arquivos prontos, que estão na pasta `materials/` do repositório.

2 A conta no GitHub

2.1 O que é o GitHub

Git e GitHub não são a mesma coisa

O **Git** é um programa de controle de versão que roda no seu computador. O **GitHub** é uma plataforma na internet que hospeda repositórios Git e acrescenta o que uma equipe precisa para colaborar: revisão de código (Pull Requests), controle de acesso, Issues, publicação de sites (GitHub Pages) e automação (GitHub Actions).

O Git funciona sozinho, sem internet. O GitHub entra quando é preciso compartilhar o trabalho, revisar o de outras pessoas ou publicar algo.

2.2 Criando a conta

1. Acesse <<https://github.com>> e clique em **Sign up**.
2. Informe e-mail, senha e nome de usuário (Figura 1). O nome de usuário aparece em todos os seus endereços, como `github.com/seu-usuario`: escolha um que você use profissionalmente.

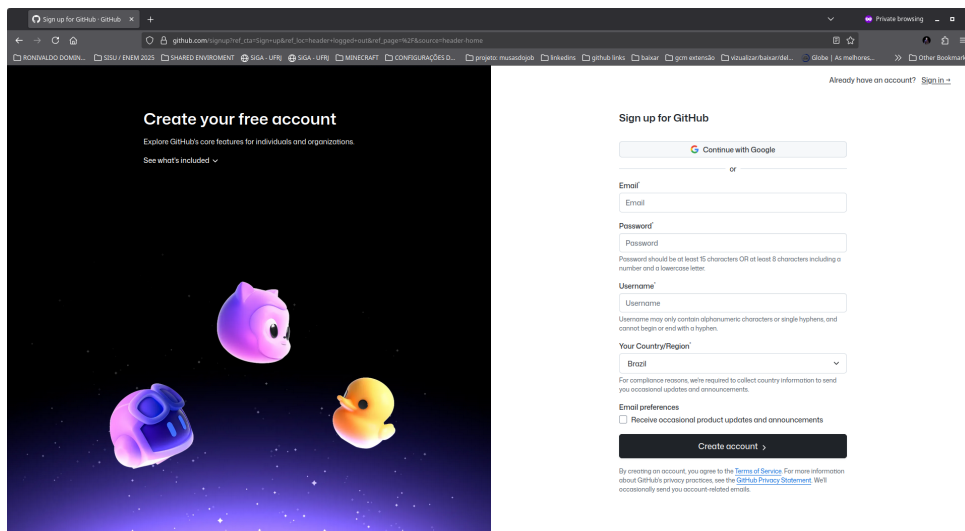


Figura 1 – Página de criação de conta no GitHub

3. Confirme o e-mail pelo código enviado e conclua as telas seguintes. Ao final, você verá a página inicial (Figura 2).

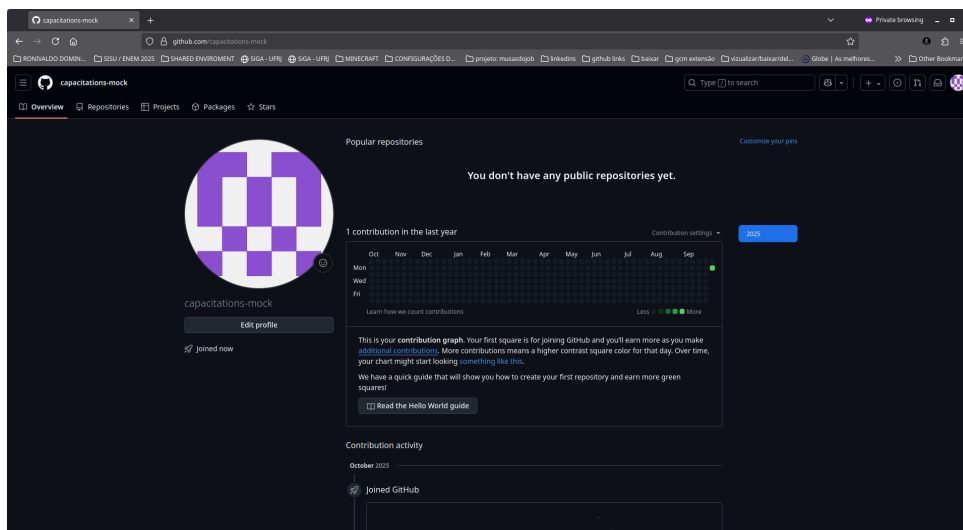


Figura 2 – Primeira visão do GitHub depois de criar a conta

2.3 Autenticação em dois fatores (2FA)

O GitHub exige a autenticação em dois fatores de quem contribui com código na plataforma. Quando a exigência chega à sua conta, você recebe um aviso e tem 45 dias para ativar; depois disso, há mais 7 dias de carência antes de o acesso ser bloqueado (GitHub 2026). Como todo participante desta capacitação vai publicar código, vale ativar logo no primeiro dia.

1. Clique na sua foto (canto superior direito) → **Settings** → **Password and authentication**.
2. Em **Two-factor authentication**, clique em **Enable two-factor authentication**.
3. Leia o QR code com um aplicativo autenticador no celular e digite o código de seis dígitos que ele mostrar.
4. **Baixe e guarde os códigos de recuperação**. Eles são a única forma de entrar na conta se você perder o celular.

Método principal e reserva

O GitHub recomenda um **aplicativo autenticador** (TOTP) como método principal e uma *passkey* ou chave de segurança como reserva. Passkeys ainda não são aceitas como método principal.

2.4 Benefícios para estudantes

2.4.1 O que o GitHub Education oferece

Estudantes verificados pelo GitHub Education recebem, sem custo e enquanto forem estudantes:

- o plano **GitHub Pro** na conta pessoal;
- o **Copilot Student**, um plano do GitHub Copilot próprio para estudantes, ativado em um passo separado (GitHub 2026);
- o **Student Developer Pack**, com ofertas de ferramentas de empresas parceiras.

Tabela 2 – Diferenças entre o GitHub Free e o GitHub Pro (conta pessoal)

Recurso	GitHub Free	GitHub Pro
Repositórios públicos e privados	ilimitados	ilimitados
Minutos de GitHub Actions por mês ^a	2.000	3.000
Armazenamento do GitHub Packages	500 MB	2 GB
Codespaces: horas de núcleo por mês	120	180
Codespaces: armazenamento por mês	15 GB	20 GB
Git LFS: armazenamento e banda por mês	10 GiB	10 GiB
GitHub Pages em repositório privado	✗	✓
Revisores obrigatórios em repositório privado	✗	✓
Suporte	comunidade	e-mail

^a Os minutos só são contados em repositórios privados: em repositórios públicos, o GitHub Actions é gratuito.

Copilot também existe no plano gratuito

Qualquer conta pode usar o **Copilot Free**, com limites mensais (por exemplo, 2.000 sugestões de código por mês). O Copilot Student libera mais recursos para quem é estudante verificado (GitHub 2026).

2.4.2 Quem pode pedir

Segundo a documentação oficial (GitHub 2026), é preciso:

- estar matriculado em um curso que confere diploma (escola, faculdade, universidade);
- ter pelo menos 13 anos;
- ter uma conta pessoal no GitHub;

- enviar um comprovante de matrícula atual: foto da carteira de estudante com a data de validade, grade de horários, histórico ou declaração de matrícula.

2.4.3 Passo a passo

1. **Cadastre o e-mail acadêmico.** Em **Settings** → **Emails**, adicione o e-mail da instituição (Figura 3) e confirme pelo link que chegar na caixa de entrada.

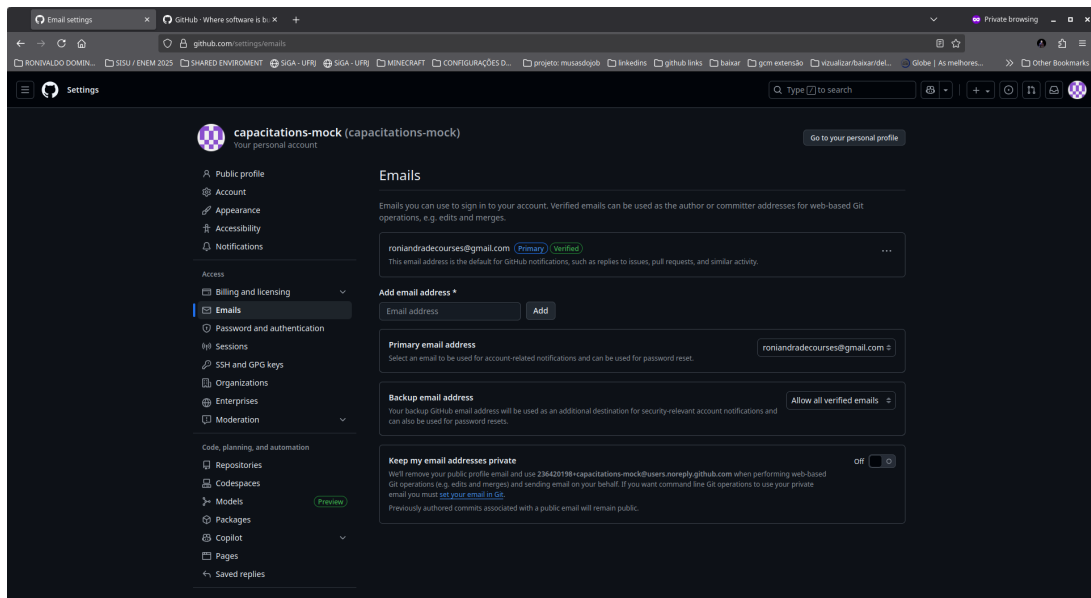


Figura 3 – Adicionando o e-mail acadêmico

2. **Abra os benefícios de educação.** Acesse <<https://github.com/settings/education/benefits>> e, em **GitHub Education**, clique em **Start an application** (Figura 5). A página pública do pack (Figura 4) leva ao mesmo formulário.

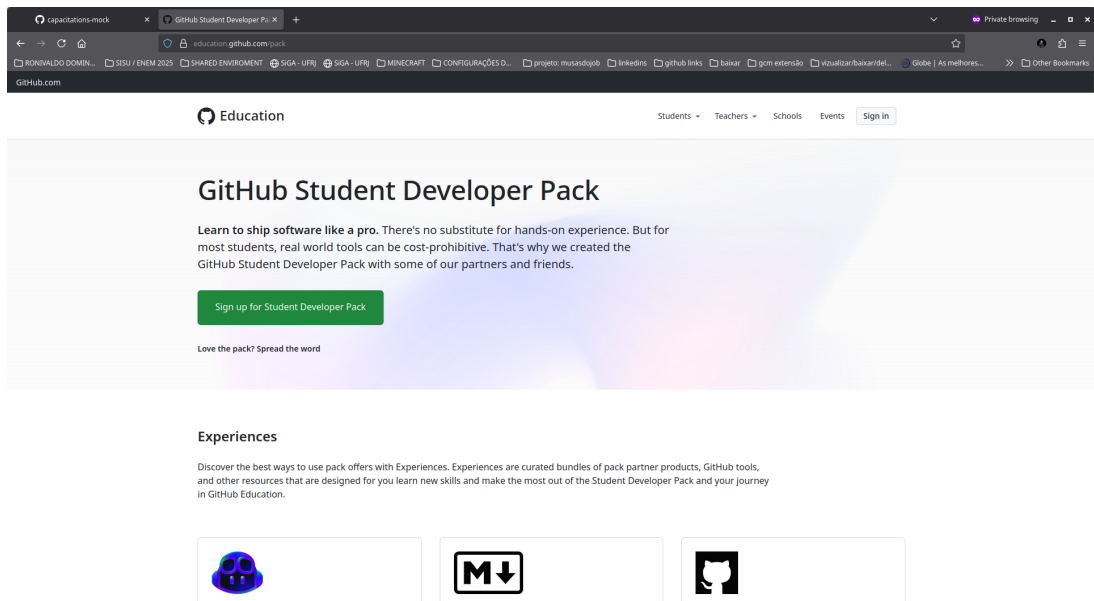


Figura 4 – Página do Student Developer Pack (`education.github.com/pack`)

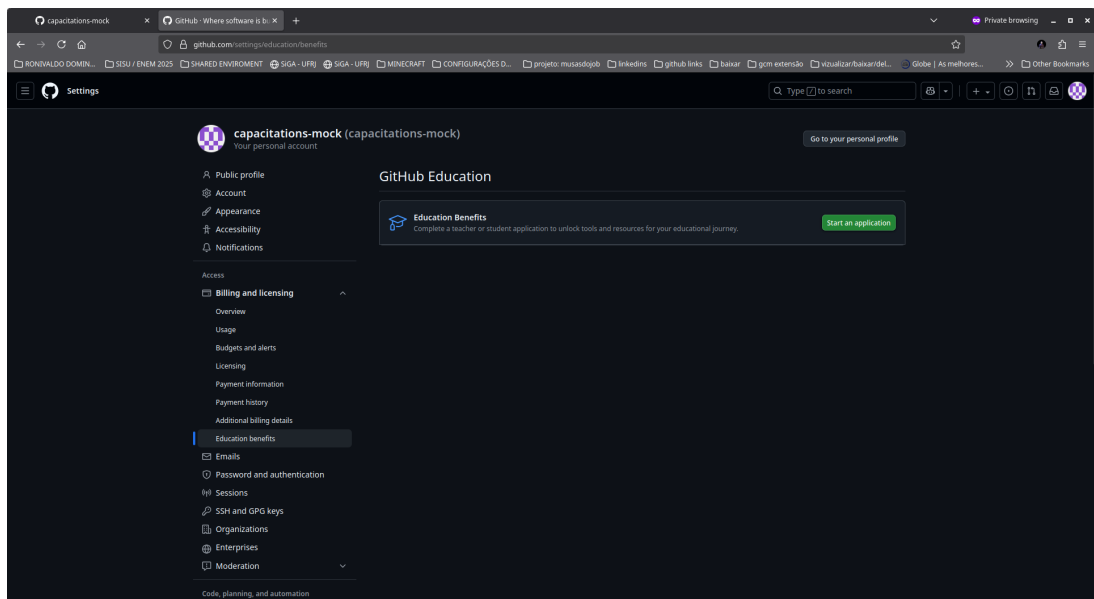


Figura 5 – **Education benefits** nas configurações, com o botão **Start an application**

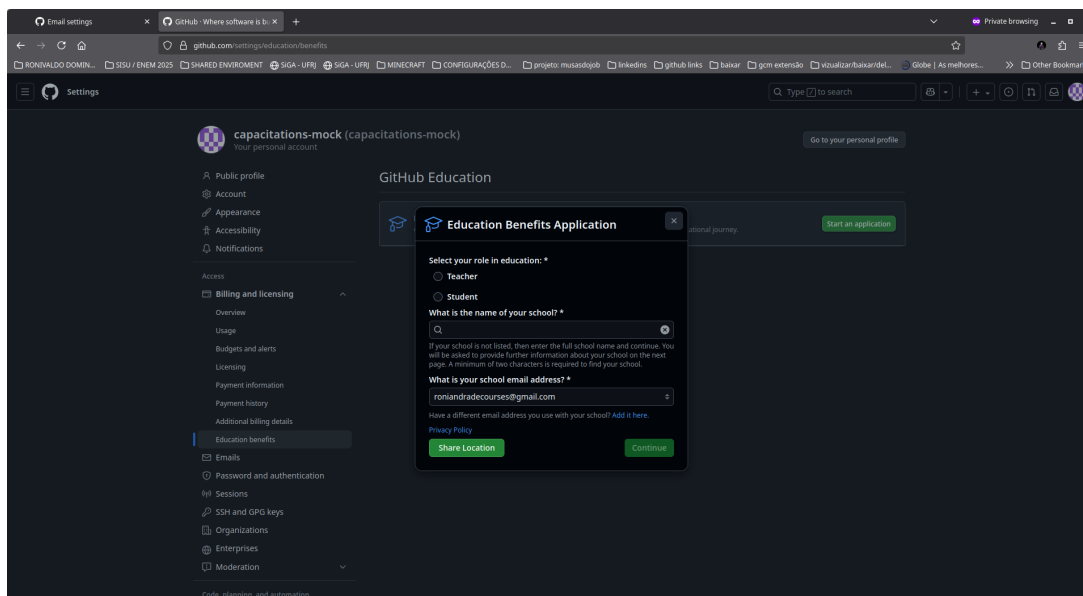


Figura 6 – Formulário da candidatura

3. **Preencha e envie.** Escolha **Student**, informe a instituição e o e-mail acadêmico (Figura 6), anexe o comprovante e clique em **Submit application**. A análise pode levar alguns dias.
4. **Ative o Copilot Student depois da aprovação.** A aprovação e a ativação do Copilot são passos separados (GitHub 2026). Volte a <https://github.com/settings/education/benefits> e siga as instruções de ativação. Se só aparecerem planos pagos, espere alguns dias: o benefício pode demorar para ser aplicado.

Experiência do autor

A foto da carteira de estudante, com nome, foto e data de validade visíveis, costuma ser analisada mais rápido que outros documentos.

As telas mudam

As capturas deste capítulo são de 2025. O GitHub reorganiza as telas com frequência; se algo estiver diferente, procure pelos nomes dos botões citados no texto.

3 Preparando o ambiente

Duas ferramentas acompanham toda a capacitação:

- o **Git**, que controla as versões no seu computador;
- o **GitHub CLI** (gh), que opera o GitHub pelo terminal: cria repositórios, abre Pull Requests e faz o login.

Siga só a seção do seu sistema.

3.1 Windows

No Windows, a instalação é feita pelo **winget**, o gerenciador de pacotes da Microsoft ([Microsoft 2026](#)). Ele vem no Windows 11 e nas versões atuais do Windows 10 (a partir da 1809), como parte do aplicativo *App Installer*.

3.1.1 Conferir o winget

Abra o **PowerShell** (menu Iniciar → digite “PowerShell”) e rode:

```
winget --version
```

Se aparecer um número de versão (Figura 7), está pronto. Se o comando não for reconhecido, abra a Microsoft Store, procure “App Installer” e atualize ou instale.

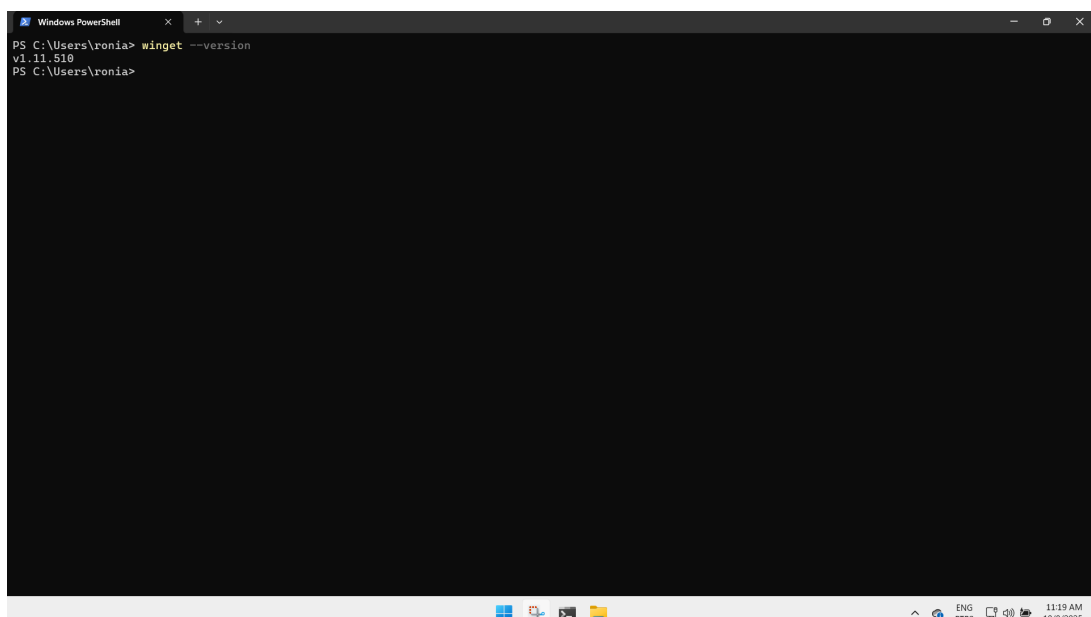


Figura 7 – Conferindo o winget

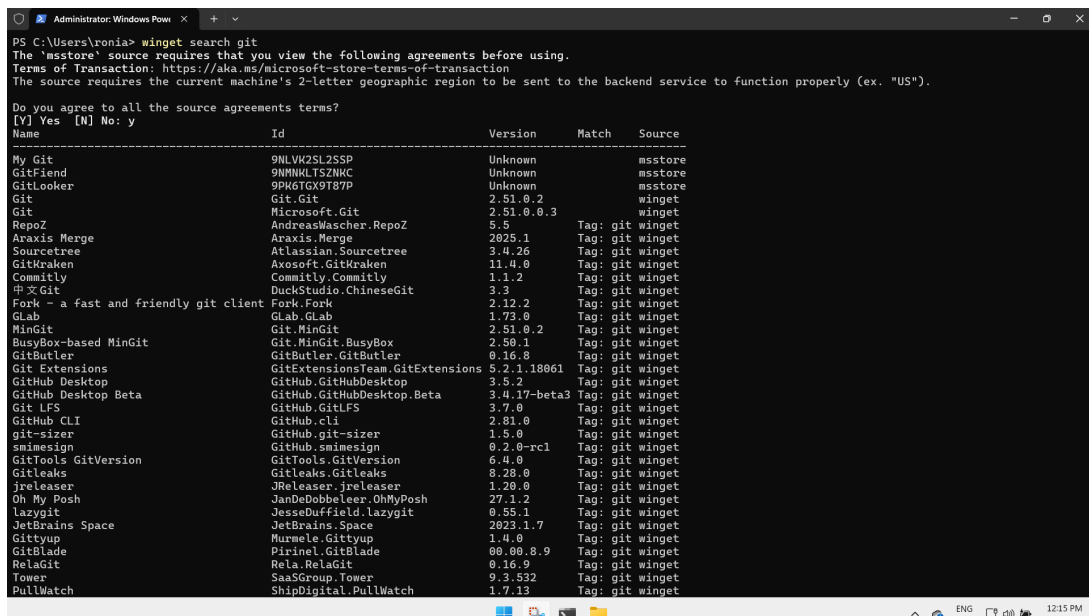
Primeiro uso

Na primeira vez, o winget pede para aceitar os termos de uso das fontes de pacotes. Digite Y e aperte `Enter`.

3.1.2 Instalar o Git

O winget search mostra o identificador (*Id*) de cada programa (Figura 8). O Id do Git é Git.Git:

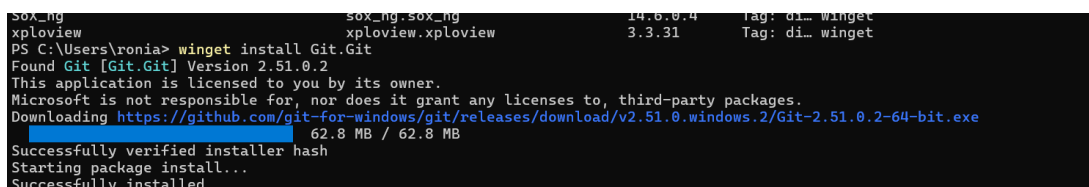
```
winget search git
winget install --id Git.Git -e --source winget
```



Name	Id	Version	Match	Source
My Git	9N1VW2SL2SSP	Unknown		msstore
GitFiend	9NMMLTSZMHC	Unknown		msstore
GitLooker	9PW6TGX9T87P	Unknown		msstore
Git	Git.Git	2.51.0.2		winget
Git	Microsoft.Git	2.51.0.0.3		winget
RepoZ	AndreasWascher.RepoZ	5.5	Tag: git	winget
Araxis Merge	Araxis.Merge	2025.1	Tag: git	winget
Sourcetree	Atlassian.Sourcetree	3.4.26	Tag: git	winget
GitKraken	Axosoft.GitKraken	11.4.0	Tag: git	winget
Commitly	Commitly.Commitly	1.1.2	Tag: git	winget
中文Git	DuckStudio.ChineseGit	3.3	Tag: git	winget
Fork - a fast and friendly git client	Fork.Fork	2.12.2	Tag: git	winget
Glab	Glab.Glab	1.73.0	Tag: git	winget
MinGit	Git.MinGit	2.51.0.2	Tag: git	winget
BusyBox-based MinGit	Git.MinGit.BusyBox	2.50.1	Tag: git	winget
GitButler	GitButler.GitButler	0.16.8	Tag: git	winget
Git Extensions	GitExtensionsTeam.GitExtensions	5.2.1.18061	Tag: git	winget
GitHub Desktop	GitHub.GitHubDesktop	3.5.2	Tag: git	winget
GitHub Desktop Beta	GitHub.GitHubDesktop.Beta	3.4.17-beta3	Tag: git	winget
Git LFS	GitHub.GitLFS	3.7.0	Tag: git	winget
GitHub CLI	GitHub.cli	2.81.0	Tag: git	winget
git-sizer	GitHub.git-sizer	1.5.0	Tag: git	winget
smimesign	GitHub.smimesign	0.2.0-rc1	Tag: git	winget
GitTools GitVersion	GitTools.GitVersion	6.4.0	Tag: git	winget
GitLeaks	GitLeaks.GitLeaks	8.28.0	Tag: git	winget
jreleaser	JReleaser.jreleaser	1.20.0	Tag: git	winget
Oh My Posh	JandDeDobbeleer.OhMyPosh	27.1.2	Tag: git	winget
Lazygit	JesseDuffield.lazygit	0.85.1	Tag: git	winget
JetBrains Space	JetBrains.Space	2023.1.7	Tag: git	winget
Gittyup	Murmele.Gittyup	1.4.0	Tag: git	winget
GitBlade	Pirinel.GitBlade	00.00.8.9	Tag: git	winget
RelaGit	Rela.RelaGit	0.16.9	Tag: git	winget
Tower	SaaSGroup.Tower	9.3.532	Tag: git	winget
PullWatch	ShipDigital.PullWatch	1.7.13	Tag: git	winget

Figura 8 – Buscando o Git no winget

A opção `-e` exige o Id exato, e `--source winget` evita a Microsoft Store. É a forma recomendada pelo site oficial do Git (Git 2026). Aguarde o fim da instalação (Figura 9).



```
sox_hy          sox_hy.sox_hy          14.0.0.4          Tag: di... winget
xplovview       xplovview.xplovview       3.3.31           Tag: di... winget
PS C:\Users\ronia> winget install Git.Git
Found Git [Git.Git] Version 2.51.0.2
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/git-for-windows/git/releases/download/v2.51.0.windows.2/Git-2.51.0.2-64-bit.exe
62.8 MB / 62.8 MB
Successfully verified installer hash
Starting package install...
Successfully installed
```

Figura 9 – Instalação do Git

O Git para Windows já traz o **Git Bash**, o terminal que usaremos daqui em diante, o **Git LFS** (capítulo 5) e o **Git Credential Manager**, que guarda o seu login com segurança.

3.1.3 Instalar o GitHub CLI

```
winget install --id GitHub.cli
```

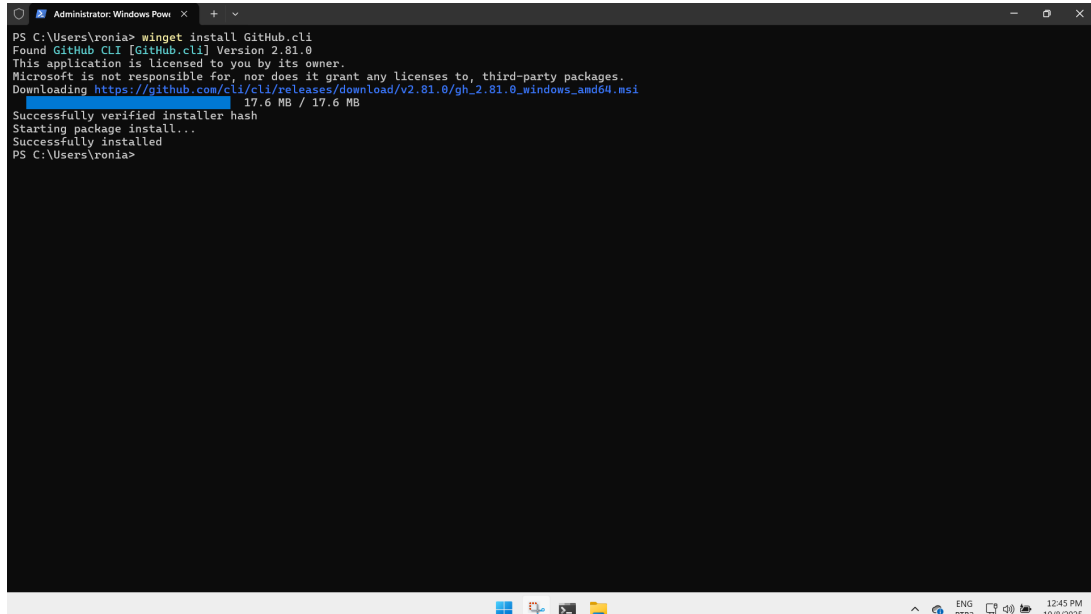


Figura 10 – Instalando o GitHub CLI

3.1.4 Conferir

Fecher o terminal e abra o Git Bash (menu Iniciar → “Git Bash”). O terminal precisa ser reaberto para enxergar os programas recém-instalados.

```
git --version  
gh --version
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ronia> git --version
git version 2.51.0.windows.2
PS C:\Users\ronia>
```

Figura 11 – Versão do Git

```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ronia> gh --version
gh version 2.81.0 (2025-10-01)
https://github.com/cli/cli/releases/tag/v2.81.0
PS C:\Users\ronia>
```

Figura 12 – Versão do GitHub CLI

Atualizar depois

Para atualizar um programa específico: `winget upgrade --id Git.Git`. Evite o `winget upgrade --all` no meio da aula: ele atualiza todos os programas do computador e pode demorar bastante.

3.2 Linux (Debian e Ubuntu)

O Git está nos repositórios da distribuição:

```
sudo apt update
sudo apt install git git-lfs
```

O GitHub CLI tem repositório próprio. Este é o comando oficial (GitHub 2026). Copie inteiro: as barras invertidas no fim das linhas unem tudo num comando só.

```
(type -p wget >/dev/null || (sudo apt update && sudo apt install wget -y)
↪ ) \
&& sudo mkdir -p -m 755 /etc/apt/keyrings \
&& out=$(mktemp) && wget -nv -O$out https://cli.github.com/packages/
↪ githubcli-archive-keyring.gpg \
&& cat $out | sudo tee /etc/apt/keyrings/githubcli-archive-keyring.gpg
↪ > /dev/null \
&& sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg \
&& sudo mkdir -p -m 755 /etc/apt/sources.list.d \
&& echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/
↪ keyrings/githubcli-archive-keyring.gpg] https://cli.github.com/
↪ packages stable main" | sudo tee /etc/apt/sources.list.d/github-
↪ cli.list > /dev/null \
&& sudo apt update \
&& sudo apt install gh -y
```

Evite o pacote gh da distribuição

Os mantenedores do GitHub CLI desaconselham o `gh` empacotado pela comunidade Debian: algumas versões antigas dele deixaram de funcionar com as APIs atuais do GitHub (GitHub 2026). Use o repositório oficial acima.

Outras distribuições (Fedora, Arch, openSUSE) estão em https://github.com/cli/cli/blob/trunk/docs/install_linux.md.

3.3 macOS

O caminho mais simples é o **Homebrew** (<https://brew.sh>). Com ele instalado:

```
brew install git git-lfs gh
```

Dica

Sem o Homebrew, rodar `git --version` no Terminal oferece instalar as *Command Line Tools* da Apple, que trazem o Git. O GitHub CLI, nesse caso, é instalado pelo pacote em <https://cli.github.com>.

3.4 Conferindo nos três sistemas

```
git --version  
gh --version  
git lfs version
```

Os três devem responder com um número de versão. Se algum comando não for reconhecido, feche e reabra o terminal. Se continuar, reinstale a ferramenta.

4 Git e GitHub CLI: configuração e login

4.1 O que é o Git

Git

O Git é um **sistema distribuído de controle de versão**, gratuito e de código aberto. Ele registra cada alteração num **commit**: uma fotografia do projeto com autor, data e uma mensagem explicando o porquê. “Distribuído” quer dizer que cada pessoa tem o histórico completo no próprio computador.

Um arquivo passa por três lugares:

Tabela 3 – As três áreas do Git

Área	O que é	Como chega lá
Diretório de trabalho	os arquivos como você os vê e edita	editando
Área de preparação (<i>staging</i>)	o que vai entrar no próximo commit	<code>git add</code>
Repositório	o histórico de commits, na pasta oculta <code>.git</code>	<code>git commit</code>

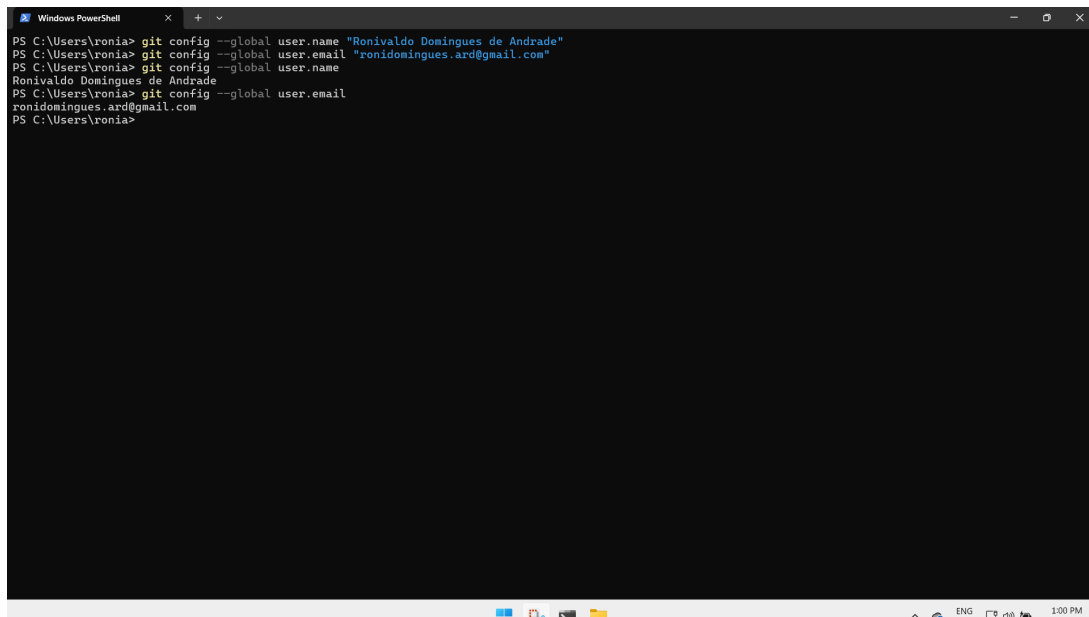
4.2 O que é o GitHub CLI

O GitHub CLI (`gh`) faz pelo terminal o que você faria no site: criar repositórios, abrir e revisar Pull Requests, acompanhar workflows. Ele também resolve a parte mais chata para iniciantes: o **login**.

4.3 Dizendo ao Git quem é você

Todo commit leva nome e e-mail. Configure uma vez por computador (Figura 13):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@exemplo.com"
```



```
PS C:\Users\ronia> git config --global user.name "Ronivaldo Domingues de Andrade"
PS C:\Users\ronia> git config --global user.email "ronidomingues.ard@gmail.com"
PS C:\Users\ronia> git config --global user.name
Ronivaldo Domingues de Andrade
PS C:\Users\ronia> git config --global user.email
ronidomingues.ard@gmail.com
PS C:\Users\ronia>
```

Figura 13 – Configurando nome e e-mail

E-mail cadastrado e privacidade

Use um e-mail cadastrado na sua conta do GitHub, para os commits aparecerem ligados ao seu perfil. Se não quiser expor o e-mail pessoal, marque **Keep my email addresses private** em **Settings** → **Emails** e use o endereço ...@users.noreply.github.com que o GitHub mostra nessa mesma tela.

Três ajustes que evitam surpresas:

```
# novos repositórios começam na branch main, como no GitHub
git config --global init.defaultBranch main

# git pull faz merge quando as branches divergem
git config --global pull.rebase false

# (opcional) VS Code como editor das mensagens de commit
git config --global core.editor "code --wait"
```

Por que o pull.rebase false

Sem essa configuração, um git pull com alterações dos dois lados termina em fatal: Need to specify how to reconcile divergent branches. O instalador do Git para Windows já define essa opção; no Linux e no macOS é preciso configurar.

Para conferir tudo: git config --global --list.

4.4 Fazendo login no GitHub

Desde agosto de 2021, o GitHub não aceita a senha da conta em operações do Git pela linha de comando. É preciso um token ou uma chave SSH. O jeito mais simples de configurar isso é o GitHub CLI.

4.4.1 Pelo GitHub CLI (recomendado)

```
gh auth login
```

Responda às perguntas assim (Figuras 14 a 18):

Tabela 4 – Respostas do `gh auth login`

Pergunta	Resposta
Where do you use GitHub?	GitHub.com
What is your preferred protocol for Git operations?	HTTPS
Authenticate Git with your GitHub credentials?	Yes
How would you like to authenticate GitHub CLI?	Login with a web browser

O terminal mostra um código de uso único. Aperte `Enter` para abrir o navegador, entre na conta, digite o código e autorize.

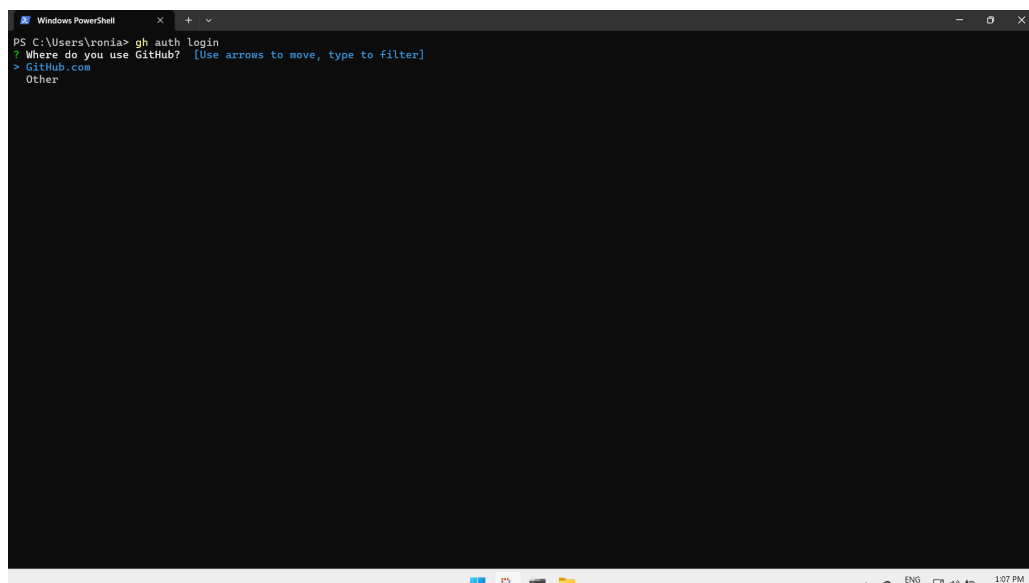


Figura 14 – `gh auth login`: escolhendo o GitHub.com

```
Windows PowerShell
PS C:\Users\ronia> gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? [Use arrows to move, type to filter]
> HTTPS
SSH
```

Figura 15 - gh auth login: escolhendo o protocolo

```
Windows PowerShell
PS C:\Users\ronia> gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? (Y/n) y
```

Figura 16 - gh auth login: autenticando o Git com as credenciais do GitHub

```
Windows PowerShell
PS C:\Users\ronia> gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? [Use arrows to move, type to filter]
> Login with a web browser
  Paste an authentication token
```

Figura 17 – gh auth login: login pelo navegador

```
Windows PowerShell
PS C:\Users\ronia> gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code:
Press Enter to open https://github.com/login/device in your browser...
```

Figura 18 – gh auth login: código de uso único

O que o “Yes” faz

Responder Yes em “Authenticate Git with your GitHub credentials?” faz o gh emprestar o login dele ao Git. É isso que faz o git push funcionar sem pedir senha. Se você respondeu No, rode `gh auth setup-git` para configurar depois (GitHub 2026).

Para conferir: `gh auth status`.

4.4.2 Por chave SSH (alternativa)

Com SSH, o computador prova quem é usando um par de chaves: a **privada** fica só com você, e a **pública** é cadastrada no GitHub.

```
# cria o par de chaves (aceite o local padrão e escolha uma senha)
ssh-keygen -t ed25519 -C "seu.email@exemplo.com"

# cadastra a chave pública na sua conta
gh ssh-key add ~/.ssh/id_ed25519.pub --title "Meu notebook"

# testa a conexão
ssh -T git@github.com
```

Na primeira conexão, o SSH pergunta se confia no servidor: responda yes. A resposta esperada é:

```
Hi seu-usuario! You've successfully authenticated, but GitHub does not
↪ provide shell access.
```

Com SSH, os endereços dos repositórios passam a ser `git@github.com:usuario/repositorio` em vez de `https://github.com/...`

4.4.3 Por token de acesso pessoal (para scripts)

Um **Personal Access Token** (PAT) é uma senha de uso restrito, com prazo de validade, que você gera no site. Para uso pessoal no terminal, o `gh auth login` já resolve. O PAT faz sentido para scripts e automações.

1. Acesse <https://github.com/settings/personal-access-tokens/new> (token **fine-grained**).
2. Dê um nome, uma validade e escolha **só os repositórios** e **só as permissões** necessárias.
3. Gere e copie o valor. Ele não aparece de novo.

Token é senha

Não compartilhe o token nem o grave em arquivos do projeto. Evite o `git config credential.helper store`: ele salva o token em texto puro no arquivo `~/.git-credentials`. O Git Credential Manager (Windows) e o `gh auth setup-git` guardam as credenciais de forma segura.

4.5 Comandos do dia a dia

Tabela 5 – Comandos básicos do Git

Comando	O que faz
<code>git init</code>	transforma a pasta atual num repositório
<code>git clone <url></code>	baixa uma cópia completa de um repositório
<code>git status</code>	mostra o que mudou e o que está preparado
<code>git add <arquivo></code>	prepara um arquivo para o próximo commit
<code>git commit -m "..."</code>	registra as alterações preparadas
<code>git log --oneline</code>	mostra o histórico, um commit por linha
<code>git switch <branch></code>	troca de branch
<code>git switch -c <branch></code>	cria uma branch e já entra nela
<code>git restore <arquivo></code>	descarta alterações não preparadas
<code>git pull</code>	baixa e integra as novidades do remoto
<code>git push</code>	envia os seus commits para o remoto
<code>git merge <branch></code>	traz as alterações de outra branch

Tabela 6 – Comandos básicos do GitHub CLI

Comando	O que faz
<code>gh auth login</code>	faz o login
<code>gh auth status</code>	mostra com qual conta você está logado
<code>gh repo create</code>	cria um repositório no GitHub
<code>gh repo clone <dono>/<repo></code>	clona um repositório pelo nome
<code>gh repo fork <dono>/<repo></code>	cria um fork na sua conta
<code>gh pr create</code>	abre um Pull Request da branch atual
<code>gh pr list</code>	lista os Pull Requests abertos
<code>gh run list</code>	mostra as execuções do GitHub Actions

As listas completas estão nos Apêndices A e B.

switch e checkout

Tutoriais antigos usam `git checkout` para trocar e criar branches. Ele ainda funciona, mas mistura funções demais. O `git switch` (para branches) e o `git restore` (para arquivos) foram criados para separar essas tarefas e são mais seguros para quem está começando.

5 Git LFS: arquivos grandes

5.1 O problema

O Git foi feito para texto, como código-fonte. Com arquivos grandes e binários (vídeos, imagens pesadas, modelos de aprendizado de máquina, arquivos .psd ou .zip), aparecem três problemas:

- **Limite do GitHub:** o GitHub avisa a partir de 50 MiB e **recusa arquivos acima de 100 MiB** num push comum (GitHub 2026). Esse limite é do GitHub, não do Git.
- **Histórico que só cresce:** binários quase não se beneficiam da forma como o Git guarda diferenças entre versões. Cada versão nova de um arquivo grande ocupa quase o tamanho inteiro de novo, e todo `git clone` baixa todas elas.
- **Operações lentas:** com muitos binários no histórico, `clone`, `fetch` e `checkout` ficam lentos.

5.2 A solução

Git LFS

O **Git LFS** (*Large File Storage*) é uma extensão do Git. No repositório fica só um **ponteiro** pequeno, de texto. O conteúdo real vai para um armazenamento separado, no próprio GitHub, e é baixado só quando necessário. Os comandos do dia a dia (`add`, `commit`, `push`) continuam os mesmos.

O ponteiro gravado no Git tem esta cara:

```
version https://git-lfs.github.com/spec/v1
oid sha256:
  ↪ e1ea497dccb8bd9d842c37f70699210c4db6d6981d7c41fec1d028d16fade26
size 200000
```

5.3 Instalação

- **Windows:** já vem com o Git para Windows.
- **Linux (Debian e Ubuntu):** `sudo apt install git-lfs`.

- **macOS:** `brew install git-lfs`.

Depois, rode **uma vez por usuário**:

```
git lfs install
```

```
Updated git hooks.  
Git LFS initialized.
```

Esse comando não instala o programa: ele liga o LFS na configuração do seu Git.

5.4 Usando

```
# diz ao LFS quais arquivos ele deve cuidar  
git lfs track "*.zip"  
git lfs track "*.psd"  
  
# a regra fica gravada no .gitattributes, que também vai para o repositório  
→ rio  
git add .gitattributes  
git add materiais.zip  
git commit -m "chore: versiona os materiais com Git LFS"  
git push  
  
# confere o que está no LFS  
git lfs ls-files
```

Configure antes de adicionar

O `git lfs track` só vale para o que for adicionado *depois*. Um arquivo grande que já entrou no histórico como arquivo comum continua lá. Mover arquivos antigos para o LFS exige `git lfs migrate`, que **reescreve o histórico** e precisa ser combinado com a equipe.

5.5 Limites e custos no GitHub

- Cada conta tem **10 GiB de armazenamento** e **10 GiB de banda** de LFS por mês, tanto no plano Free quanto no Pro (GitHub 2026). Acima disso, o uso é cobrado.
- O tamanho máximo de um arquivo no LFS é de 2 GB nos planos Free e Pro.

- **Apagar um arquivo do LFS não libera espaço** sozinho. Para remover de verdade, é preciso reescrever o histórico ou apagar o repositório.

Quando não usar

Não use o LFS para arquivos pequenos em grande quantidade nem para o que pode ser gerado de novo (arquivos compilados, dependências): esses devem ir para o `.gitignore`.

6 Commits, merges e Pull Requests

6.1 Introdução

Commits, merges e pull requests são elementos centrais no fluxo de trabalho com Git e plataformas como GitHub. Um **commit** é uma unidade lógica de alteração no repositório; um **merge** integra mudanças de uma branch em outra; e um **pull request** (PR) é uma solicitação formal de revisão e integração de uma branch — normalmente usada em colaboração para revisar código, executar checks automáticos (CI) e registrar a decisão de integrar.

6.2 Commits

6.2.1 O que é um commit

Um commit registra o estado do diretório de trabalho (os arquivos staged) em um nó do histórico do Git. Cada commit tem um identificador (SHA), meta-data (autor, data) e uma mensagem que descreve a mudança.

6.2.2 Boas práticas de commits

- Faça commits atômicos: cada commit deve representar uma *única* mudança lógica.
- Mensagens claras: use assunto imperativo curto (~50 caracteres) + linha em branco + corpo explicativo se necessário (72 caracteres por linha).
- Inclua referência a issues quando relevante (ex.: “Closes #42”).
- Evite commitar arquivos gerados (binários, dependências) — use `.gitignore`.
- Assine os commits (capítulo 9).

6.2.3 Fazendo commits — passo a passo

1. Verificar o estado dos arquivos:

```
git status
```

2. Ver as mudanças não staged:

```
git diff          # diferenças não adicionadas ao stage
git diff --staged # diferenças preparadas para commit
```

3. Adicionar alterações ao stage (todo arquivo ou interativo):

```
git add caminho/arquivo.txt # adiciona arquivo específico
git add .                    # adiciona tudo (cuidado)
git add -p                   # adiciona parcialmente (patch)
```

4. Criar o commit com boa mensagem:

```
git commit -m "Assunto curto em imperativo"
# ou para mensagem longa (editor):
git commit
```

Exemplo de mensagem:

Corrige cálculo de juros

Ajusta a fórmula de cálculo para considerar juros compostos quando o período é maior que 12 meses. Testes unitários adicionados para cobrir casos de fronteira.

5. Ver histórico resumido:

```
git log --oneline --graph --decorate --all
```

6.2.4 Editar o último commit / corrigir mensagens

```
# alterar o conteúdo do último commit (já staged)
git commit --amend

# alterar apenas a mensagem do último commit
git commit --amend -m "Nova mensagem"
```

Atenção: se o commit já foi enviado ao remoto, evite `--amend` sem combinar com a equipe — ele reescreve o histórico e exigirá push forçado.

6.2.5 Desfazer / alterar staging

[illegible]

```
git reset --soft HEAD~1      # desfaz último commit, mantendo
                              # mudanças staged

git reset --mixed HEAD~1     # desfaz último commit, mantendo
                              # mudanças no working tree
                              # (unstaged)

git reset --hard HEAD~1      # desfaz último commit e
                              # descarta mudanças (CUIDADO)
```

Como escrever a mensagem

O capítulo 7 mostra um padrão para as mensagens de commit, o *Conventional Commits*.

6.3 Merges

6.3.1 Tipos de merge

Fast-forward Se a branch destino não avançou desde que a feature foi criada, o Git apenas avança o ponteiro (sem novo commit de merge).

Merge commit Cria um commit de merge que documenta a união de dois históricos (útil para preservar contexto de branch).

Rebase Reaplica commits de uma branch sobre outra, produzindo um histórico linear (reduz “ruído” dos merges, mas reescreve histórico).

6.3.2 Merge local com merge commit (passo a passo)

1. Atualize a branch principal:

```
git switch main
git pull origin main
```

2. Mudar para a branch de feature (se necessário):

```
git switch feature/minha-feature
git pull origin feature/minha-feature
```

3. Voltar para a branch de destino e fazer merge:

```
git switch main
git merge --no-ff feature/minha-feature
# --no-ff força um commit de merge,
```

```
# preservando histórico da branch
```

4. Caso não haja conflitos, o Git criará o commit de merge automaticamente. Em caso de conflitos, siga a seção "Resolver conflitos" abaixo.
5. Envie as mudanças para o remoto:

```
git push origin main
```

6.3.3 Rebase (passo a passo) — para um histórico linear

1. Atualize a base:

```
git switch main  
git pull origin main
```

2. Rebase da feature sobre a main:

```
git switch feature/minha-feature  
git rebase main
```

3. Resolva conflitos (se aparecerem), use `git rebase --continue` e, ao final:

```
# force push (com cuidado) após reescrever histórico  
git push --force-with-lease origin feature/minha-feature
```

4. **Observação:** reescrever histórico (rebase + push forçado) requer coordenação com outros colaboradores.

6.3.4 Resolver conflitos — passo a passo

1. Ao encontrar conflito durante merge/rebase, o Git mostra arquivos conflitantes:

```
CONFLICT (content): Merge conflict in caminho/arquivo.txt
```

2. Abra o arquivo e localize os marcadores:

```
<<<<<< HEAD  
conteúdo na branch atual (main)  
=====  
conteúdo vindo da branch feature/minha-feature  
>>>>>> feature/minha-feature
```

3. Edite o arquivo para a versão desejada (manter, combinar ou reescrever o trecho).

4. Marque o conflito como resolvido:

```
git add caminho/arquivo.txt
# se for rebase:
git rebase --continue
# se for merge:
git commit # caso o Git não tenha criado o commit
           ↳ automaticamente
```

5. Se quiser abortar a operação:

```
git merge --abort # durante um merge
git rebase --abort # durante um rebase
```

6.3.5 Squash e reescrita de commits (passo a passo)

1. Rebase interativo para combinar commits locais:

```
git switch feature/minha-feature
git rebase -i main
```

2. No editor que abre, marque squash (ou s) nos commits que deseja unir ao commit anterior. Salve e feche.

3. Após o rebase, force push com segurança:

```
git push --force-with-lease origin feature/minha-feature
```

4. Use --force-with-lease em vez de --force quando possível — ele evita sobrescrever pushes alheios.

6.4 Pull Requests (PR)

6.4.1 O que é um Pull Request

Um PR é uma solicitação para integrar as mudanças de uma branch em outra (normalmente de uma branch de feature para main ou develop) e serve como ponto central para revisão de código, execução de pipelines de CI e documentação da mudança.

6.4.2 Fluxo básico — criando um PR (via web)

1. Crie uma branch local para a sua feature:

```
git switch -c feature/minha-feature
```

2. Faça commits locais e envie a branch para o remoto:

```
git push -u origin feature/minha-feature
```

3. No repositório do GitHub, acesse Pull requests → New pull request.

4. Selecione a **branch de origem** (feature/minha-feature) e a **branch de destino** (ex.: main).

5. Preencha o título e a descrição: explique o **porquê** e o **o que** foi alterado. Use referências a issues (ex.: “Closes #123”).

6. Configure revisores, labels, milestone e assignees.

7. Se ainda não está pronta, crie como **Draft pull request** (rascunho).

8. Aguarde revisão, corrija comentários fazendo novos commits na mesma branch e push — eles serão anexados automaticamente ao PR.

9. Quando aprovado, escolha a estratégia de merge (merge commit / squash and merge / rebase and merge) e realize o merge.

6.4.3 Criar e gerenciar PRs via GitHub CLI (passo a passo)

```
# autentique (uma vez)
gh auth login

# depois de push da branch
gh pr create --base main --head feature/minha-feature --title "Título do
  ↳ PR" --body "Descrição detalhada" --reviewer usuario1,usuario2

# abrir PR como draft:
gh pr create --draft --base main --head feature/minha-feature --fill

# listar PRs locais:
gh pr list

# fechar ou mesclar via CLI:
gh pr merge <numero-ou-url> --squash --delete-branch
# ou
gh pr merge <numero> --merge          # cria commit de merge
gh pr merge <numero> --rebase         # rebase and merge
```

6.4.4 Checklist para revisão de Pull Request

- O PR tem um título e descrição claros (o *porquê* e o *o que*);
- Testes automatizados adicionados/atualizados e pipeline CI passando;
- Código segue padrões de lint e estilo;
- Mudanças pequenas e focadas (um PR por responsabilidade);
- Documentação e comentários quando necessário;
- Evidências visuais (screenshots) quando há alterações de UI.

6.4.5 Depois do merge — limpeza e sincronização

```
# atualizar a branch principal local
git switch main
git pull origin main

# remover branch remota (após merge)
git push origin --delete feature/minha-feature

# remover branch local
git branch -d feature/minha-feature

# caso a branch não possa ser deletada por não estar totalmente mesclada:
git branch -D feature/minha-feature
```

Também é útil rodar:

```
git fetch --prune
```

para remover referências remotas deletadas.

6.5 Boas práticas e recomendações finais

- Mantenha PRs pequenos e revisáveis; grandes PRs demoram mais para receber feedback.
- Automatize checks com CI (testes, lint, análise estática) e impeça merge enquanto falharem (branch protection).
- Use `--force-with-lease` quando precisar reescrever histórico; nunca force sem checar se colegas não empurraram commits.
- Documente o fluxo do time (merge strategy preferida — ex.: squash para commits limpos, merge commit para histórico preservado).

- Escreva mensagens de commit úteis e mantenha um padrão no time (ex.: Conventional Commits) para facilitar geração automática de changelogs.

6.6 Exemplos rápidos de comandos úteis

```
# ver status e diferenças
git status
git diff
git diff --staged

# histórico e log
git log --oneline --graph --decorate --all

# adicionar parcialmente
git add -p

# rebase interativo (últimos 3 commits)
git rebase -i HEAD~3

# desfazer mudanças locais (trabalhe com cuidado)
git restore arquivo.txt
git reset --hard HEAD

# forçar push com segurança
git push --force-with-lease origin minha-branch
```

7 Padrões de commit

No capítulo 6 vimos como fazer commits. Este capítulo trata de **como escrever essa mensagem** de um jeito que a equipe inteira (e as ferramentas) consigam ler.

7.1 Por que padronizar

Compare dois históricos do mesmo projeto:

```
a41aded ajustes
c2524a8 arrumei
afe16bb agora vai
4203776 mudanças do joão
```

```
a41aded fix(login): corrige senha vazia
c2524a8 feat(relatorio): exporta em PDF
afe16bb docs: explica a instalação
4203776 test(login): cobre senha vazia
```

No segundo, dá para saber **o tipo** da mudança, **onde** ela aconteceu e **o que** foi feito, sem abrir um arquivo sequer. Um padrão de commit também permite automatizar tarefas: gerar notas de versão, decidir o próximo número de versão e barrar mensagens fora do combinado.

A regra mais importante

O melhor padrão é **o que a equipe combinou e segue**. Os padrões deste capítulo são os mais usados, mas consistência vale mais que qualquer escolha isolada.

7.2 Conventional Commits

O padrão mais difundido é o **Conventional Commits 1.0.0** (<<https://www.conventionalcommits.org/pt-br/v1.0.0/>>). A estrutura é:

```
<tipo>[escopo opcional]: <descrição>

[corpo opcional]

[rodapé(s) opcional(is)]
```

As regras principais da especificação:

- o **tipo** é obrigatório e vem primeiro, seguido de **dois-pontos e espaço**;
- o **escopo** é opcional: um substantivo entre parênteses que diz qual parte do sistema mudou, como `fix(parser):`;
- a **descrição** vem logo depois de `:` e resume a mudança;
- o **corpo** é opcional e começa **uma linha em branco** depois da descrição, com quantos parágrafos forem necessários;
- os **rodapés** são opcionais e vêm uma linha em branco depois do corpo, no formato `Token: valor (por exemplo, Refs: #123 ou Reviewed-by: Maria)`;
- uma **quebra de compatibilidade** (*breaking change*) é indicada com **!** antes dos dois-pontos, com um rodapé `BREAKING CHANGE: <explicação>`, ou com os dois.

```
feat(api)!: exige autenticação em todos os endpoints
```

Antes, os endpoints de consulta eram públicos. Agora todo pedido precisa do cabeçalho `Authorization` com um token válido.

`BREAKING CHANGE:` clientes sem token passam a receber 401.

`Refs:` [#42](#)

7.2.1 Os tipos

A especificação só define dois tipos: `feat` (novo recurso) e `fix` (correção de bug). Os outros são convenção, herdada das regras do projeto Angular. A lista abaixo é a aceita pela configuração padrão do *commitlint* (seção 7.6):

Tipo	Quando usar
feat	um recurso novo para quem usa o sistema
fix	a correção de um bug
docs	só documentação (README, comentários, manuais)
style	formatação que não muda o comportamento (espaços, ponto e vírgula, indentação)
refactor	reorganização do código sem mudar o que ele faz nem corrigir bug
perf	melhoria de desempenho
test	criação ou ajuste de testes
build	sistema de build ou dependências (npm, pip, Maven...)
ci	configuração de integração contínua (GitHub Actions)
chore	tarefas de manutenção que não se encaixam nos outros tipos
revert	desfaz um commit anterior

Tipos e versões

Com Conventional Commits, ferramentas conseguem calcular a próxima versão pelo *Versionamento Semântico*: `fix` sobe o número de correção (**1.2.3**), `feat` sobe o número de recurso (**1.3.0**) e um `BREAKING CHANGE` sobe o número principal (**2.0.0**).

Tipos fora da lista

Algumas referências populares em português acrescentam tipos como `raw`, `cleanup`, `remove` e `init`. Eles não fazem parte da especificação nem da configuração padrão do commitlint, que os rejeita. Só use se a equipe combinar e configurar isso.

7.3 Boas práticas para a mensagem

- **Uma mudança por commit.** Se a descrição precisa de um “e”, provavelmente são dois commits. Commits pequenos são mais fáceis de revisar e de desfazer.
- **Descrição curta.** O livro *Pro Git* recomenda uma primeira linha de até uns 50 caracteres e o corpo quebrado em 72 colunas. A configuração padrão do commitlint aceita até 100 caracteres no cabeçalho.

- **O corpo explica o porquê.** O “o quê” está no `git diff`. O motivo da mudança só existe na mensagem.
- **Um único modo verbal.** As convenções em inglês usam o imperativo (“add”, “fix”). Em português, equipes usam o imperativo (“corrigir”), o presente (“corrigir”) ou o gerúndio (“corrigindo”). A especificação não exige nenhum. Escolha um e mantenha.
- **Idioma combinado.** A especificação também não fixa idioma. Misturar português e inglês no mesmo histórico atrapalha a busca.

7.4 Gitmoji: emojis nas mensagens

O **gitmoji** (<<https://gitmoji.dev>>) associa um emoji a cada tipo de mudança. No texto do commit, o emoji aparece como um código entre dois-pontos, o *shortcode*. Alguns dos mais usados, com a descrição oficial traduzida:

Código	Significado	Código	Significado
:sparkles:	novo recurso	:bug:	corrige um bug
:memo:	documentação	:art:	estrutura ou formato do código
:zap:	desempenho	:recycle:	refatoração
:fire:	remove código ou arquivos	:truck:	move ou renomeia arquivos
:white_check_mark:	testes	:construction_worker:	CI
:wrench:	arquivos de configuração	:package:	arquivos compilados ou pacotes
:lock:	segurança ou privacidade	:ambulance:	correção crítica urgente
:pencil2:	corrige erro de digitação	:rewind:	reverte mudanças
:tada:	começa um projeto	:bookmark:	versão ou tag
:page_facing_up:	licença	:card_file_box:	mudanças de banco de dados

A lista completa tem 75 emojis e está em <<https://gitmoji.dev>>.

Três cuidados com emojis

1. **O terminal não converte.** O GitHub mostra o emoji na página do commit, mas o `git log` exibe o texto `:sparkles:` como foi escrito.
2. **O mesmo emoji muda de sentido entre fontes.** Na lista oficial, `:boom:` é quebra de compatibilidade, mas há tutoriais que o usam para “reverter”. Siga a lista oficial.
3. **Emoji antes do tipo quebra o padrão.** A especificação exige que a mensagem comece pelo tipo. Uma mensagem como `:bug: fix: corrige o login` é rejeitada pelo `commitlint` na configuração padrão.

7.5 Um padrão combinando os dois

Muitas equipes juntam gitmoji e Conventional Commits num padrão próprio. É o caso dos repositórios do autor deste guia, que usam:

```
<:emoji:> <tipo>(<escopo>): <Descrição no gerúndio>

:memo: docs(guia): Adicionando os exercícios de assinatura
:bug: fix(login): Corrigindo o acesso com senha vazia
:truck: chore(docs): Movendo o manual para a pasta materials
```

Não há nada de errado nisso, desde que **seja uma decisão da equipe, esteja escrita** (no README ou num guia de contribuição) e **as ferramentas estejam configuradas para aceitá-la**. O ponto é a consistência.

7.6 Ferramentas

`commitlint`: barrar mensagens fora do padrão

O **commitlint** (<https://commitlint.js.org>) confere cada mensagem antes de o commit ser gravado. Num projeto com Node.js:

```
npm install -D @commitlint/cli @commitlint/config-conventional
echo "export default { extends: ['@commitlint/config-conventional'] };" >
  ↳ commitlint.config.js

# testar uma mensagem sem precisar fazer commit
echo "feat: adiciona exportação em PDF" | npx commitlint
```

Para rodar automaticamente em todo `git commit`, o guia oficial usa o *husky*.

```
npm install --save-dev husky
npx husky init
```

```
echo "npx --no -- commitlint --edit \$1" > .husky/commit-msg
```

Depois disso, `git commit -m "foo: teste"` é recusado, porque `foo` não é um tipo válido.

A configuração padrão é rígida

Além de exigir o tipo no início, a configuração padrão rejeita descrições que começam com letra maiúscula. Um padrão como o da seção anterior precisa de regras próprias no `commitlint.config.js`.

`gitmoji-cli`: escolher o emoji num menu

```
npm i -g gitmoji-cli
gitmoji -c          # monta o commit de forma interativa
gitmoji -l          # lista os emojis
```

Sem ferramenta nenhuma

Nada disso é obrigatório. Um arquivo `CONTRIBUTING.md` com o padrão e três exemplos já resolve boa parte do problema, e a revisão do Pull Request (capítulo 6) é um bom momento para lembrar o combinado.

8 GitHub Pages e GitHub Actions

8.1 GitHub Pages: um site a partir do repositório

O **GitHub Pages** hospeda **sites estáticos** (HTML, CSS e JavaScript) direto de um repositório, de graça, no endereço `https://<seu-usuario>.github.io/<repositorio>/` (GitHub 2026).

- Em repositório **público**, funciona em qualquer conta. Em repositório **privado**, exige o GitHub Pro (que vem com os benefícios de estudante, capítulo 2).
- Aceita domínio próprio, além do `github.io`.
- Não roda código no servidor: nada de PHP, banco de dados ou Python do lado do servidor.

8.1.1 As duas formas de publicar

A escolha fica em **Settings** → **Pages** → **Build and deployment** → **Source**:

Tabela 7 – Fontes de publicação do GitHub Pages

Source	Quando usar
Deploy from a branch	o site já está pronto no repositório: basta escolher a branch (por exemplo, <code>main</code>) e a pasta (<code>/(root)</code> ou <code>/docs</code>)
GitHub Actions	o site precisa ser gerado antes de publicar, ou você quer controlar o que é publicado. É o caminho recomendado pela documentação atual

O `index.html` precisa estar na pasta publicada

O Pages abre o `index.html` da pasta escolhida. Se ele estiver dentro de uma subpasta (por exemplo, `jogo-da-memoria/index.html`), o endereço principal responde com erro 404.

8.2 GitHub Actions: automação dentro do GitHub

O **GitHub Actions** executa tarefas automaticamente quando algo acontece no repositório: rodar testes a cada push, compilar um projeto, publicar um site (GitHub 2026).

Tabela 8 – Vocabulário do GitHub Actions

Termo	O que é
Workflow	o processo automatizado, descrito num arquivo YAML em <code>.github/workflows/</code>
Evento (on)	o que dispara o workflow: um push, um Pull Request, um horário, um clique manual (<code>workflow_dispatch</code>)
Job	um conjunto de passos que roda numa mesma máquina
Step	um passo: um comando (<code>run</code>) ou uma ação pronta (<code>uses</code>)
Runner	a máquina virtual do GitHub que executa o job, como <code>ubuntu-latest</code>
Action	um passo pronto e reutilizável, como <code>actions/checkout</code>

Minutos

Em repositórios **públicos**, o GitHub Actions é gratuito. Em privados, os minutos por mês dependem do plano (tabela 2).

8.2.1 Exemplo 1: publicar um site pelo Actions

Salve como `.github/workflows/pages.yml` e, em **Settings** → **Pages**, escolha a Source **GitHub Actions**:

```
name: Publicar o site

on:
  push:
    branches: [ main ]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  publicar:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deploy.outputs.page_url }
    steps:
      - uses: actions/checkout@v7
```

```

- uses: actions/configure-pages@v6
- uses: actions/upload-pages-artifact@v5
  with:
    path: .
- id: deploy
  uses: actions/deploy-pages@v5

```

O path: . publica a raiz do repositório. Se o site estiver numa subpasta, troque pelo caminho dela.

8.2.2 Exemplo 2: rodar um script e guardar o resultado

Este workflow roda o `main.py` (disponível em `materials/`), que cria um arquivo `resultado.txt`, e grava esse arquivo de volta no repositório:

```

name: Executar o script Python

on:
  push:
    branches: [ main ]
  workflow_dispatch:

permissions:
  contents: write # necessário para o git push no último passo

jobs:
  executar:
    runs-on: ubuntu-latest
    steps:
      # 1. Baixa o repositório para a máquina do GitHub
      - uses: actions/checkout@v7

      # 2. Instala o Python
      - uses: actions/setup-python@v7
        with:
          python-version: '3.x'

      # 3. Roda o script, que cria o resultado.txt
      - name: Executar o main.py
        run: python main.py

      # 4. Grava o resultado no repositório
      - name: Commitar o resultado
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "github-actions[bot]@users.noreply.github
          ↪ .com"

```

```
git add resultado.txt
git commit -m "chore: atualiza o resultado.txt" || echo "Nada
    ↳ para commitar"
git push
```

Sem permissão, o push falha

Por padrão, o token que o workflow recebe (GITHUB_TOKEN) pode ser só de leitura. Sem o `permissions: contents: write`, o `git push` do último passo falha com erro 403.

8.2.3 E o `[skip ci]`?

Poderia parecer que o push feito pelo próprio workflow dispara o workflow de novo, num laço infinito. **Não dispara:** pushes feitos com o GITHUB_TOKEN não iniciam novos workflows. O `[skip ci]` na mensagem do commit (ou `[ci skip]`, `[no ci]`, `[skip actions]`) só é necessário quando o push é feito com um token pessoal ou de aplicativo (GitHub 2026).

8.2.4 Acompanhando

A aba **Actions** do repositório mostra cada execução, com o log de cada passo. Pelo terminal:

```
gh run list          # últimas execuções
gh run watch         # acompanha a execução em andamento
```

9 Assinatura de commits com GPG e SSH

9.1 O problema: o autor de um commit é só um texto

No capítulo 4 configuramos o nome e o e-mail com `git config`. O Git aceita **qualquer** valor ali, sem conferir nada. Veja:

```
git -c user.name="Linus Torvalds" -c user.email="torvalds@linux-  
↳ foundation.org" \  
    commit --allow-empty -m "Commit que parece do Linus"  
git log --format='%an <%ae>' -1
```

```
Linus Torvalds <torvalds@linux-foundation.org>
```

O commit foi feito por nós, mas o histórico diz que foi o criador do Git. Se esse commit for enviado ao GitHub, ele aparece com o nome e a foto de quem tem aquele e-mail cadastrado.

Assinatura de commit

Assinar um commit é anexar a ele uma **assinatura criptográfica** feita com uma chave privada que só você tem. Quem tem a sua chave **pública** consegue conferir que o commit veio de você e que não foi alterado depois. O GitHub faz essa conferência e mostra um selo ao lado do commit.

Os selos

Selo	Significado
Verified	o commit está assinado e a assinatura foi conferida com uma chave cadastrada na conta
Unverified	o commit está assinado, mas a assinatura não pôde ser conferida (chave não cadastrada, e-mail diferente, chave de outra pessoa)
(sem selo)	o commit não foi assinado

O e-mail precisa bater

Para o selo **Verified**, o e-mail do commit (`git config user.email`) precisa ser um e-mail **verificado** na sua conta do GitHub. No caso do GPG, ele também precisa estar dentro da chave. Essa é a causa mais comum de “Unverified”.

9.2 GPG ou SSH?

Há dois jeitos de assinar, e os dois são aceitos pelo GitHub:

	GPG	SSH
Ferramenta	GnuPG (<code>gpg</code>)	OpenSSH (<code>ssh-keygen</code>), que já vem com o Git
Requisito	GnuPG instalado	Git 2.34 ou mais novo
Chave	uma chave própria, que carrega nome e e-mail	a mesma chave SSH do capítulo 4 pode servir
Validade	pode ter data de expiração embutida	sem data de expiração
Conferir no terminal	direto, com o chaveiro do GPG	precisa de um arquivo <code>allowed_signers</code>
Quando escolher	padrão consolidado, usado há muito tempo	mais simples para quem já usa SSH

Dica

Escolha **um** dos dois e siga só a seção correspondente. Se não souber qual, comece pelo SSH: são menos passos.

9.3 Assinando com GPG

9.3.1 Instalar o GnuPG

- **Linux:** costuma vir instalado. Confira com `gpg --version`. No Debian e no Ubuntu, se faltar: `sudo apt install gnupg`.
- **Windows:** o Git for Windows já traz um `gpg` dentro do **Git Bash**. Use o Git Bash para todos os comandos desta seção.

Windows: um GPG só

Se você também instalar o Gpg4win, passa a ter **dois** GPGs no computador, cada um com o próprio chaveiro. Uma chave criada num não aparece no outro, e o Git reclama de “No secret key”. Crie a chave e assine sempre pelo mesmo. Para saber qual o Git está usando, rode `which gpg` no Git Bash.

9.3.2 Passo 1 — Ver se você já tem uma chave

```
gpg --list-secret-keys --keyid-format=long
```

Se aparecer uma chave com o seu e-mail, pule para o passo 3.

9.3.3 Passo 2 — Gerar a chave

```
gpg --full-generate-key
```

O comando faz algumas perguntas:

Pergunta	O que responder
Tipo de chave	aperte Enter para aceitar o padrão. Nas versões novas é ECC (ed25519); nas antigas, “RSA e RSA”. O GitHub aceita os dois
Tamanho	só aparece para RSA: digite 4096
Validade	0 = não expira, que é o padrão recomendado pelo GitHub. Uma data (como 1y, um ano) é mais segura, mas exige renovar a chave
Nome e e-mail	o seu nome e o mesmo e-mail verificado no GitHub
Senha	uma senha forte. Ela protege a chave se alguém copiar o seu computador

9.3.4 Passo 3 — Descobrir o ID da chave

```
gpg --list-secret-keys --keyid-format=long
```

```
sec  rsa4096/30F2B65B9246B6CA 2017-08-18 [SC]
      D5E4F29F3275DC0CDA8FFC8730F2B65B9246B6CA
uid                               [ultimate] Maria Exemplo <maria@exemplo.com>
ssb  rsa4096/B7ABC0813E4028C0 2017-08-18 [E]
```

O ID é o que vem **depois da barra** na linha `sec`: aqui, 30F2B65B9246B6CA. Numa chave ECC, a linha começa com ed25519/. O seu ID será outro.

9.3.5 Passo 4 — Exportar a chave pública

```
gpg --armor --export 30F2B65B9246B6CA
```

Copie **tudo**, da linha `-----BEGIN PGP PUBLIC KEY BLOCK-----` até a linha `-----END PGP PUBLIC KEY BLOCK-----`, incluindo as duas.

Pública, nunca privada

O que vai para o site é a chave **pública** (`--export`). Nunca use `--export-secret-keys` para isso. A chave privada não sai do seu computador.

9.3.6 Passo 5 — Cadastrar no GitHub

1. Clique na sua foto (canto superior direito) → **Settings**.
2. Na seção **Access**, clique em **SSH and GPG keys**.
3. Clique em **New GPG key**.
4. Em **Title**, dê um nome (por exemplo, “Notebook pessoal”). Em **Key**, cole o bloco copiado.
5. Clique em **Add GPG key**. O GitHub pode pedir a sua senha de novo.

9.3.7 Passo 6 — Dizer ao Git qual chave usar

```
git config --global user.signingkey 30F2B65B9246B6CA
git config --global commit.gpgsign true           # assina todo commit
git config --global tag.gpgsign true               # assina toda tag
```

Já usou assinatura SSH antes?

Se algum dia configurou `gpg.format ssh`, desfaça com `git config --global --unset gpg.format`. Sem isso, o Git continua tentando assinar com SSH.

9.3.8 Passo 7 — Ensinar o GPG a pedir a senha no terminal

No Linux e no Git Bash, o GPG precisa saber em qual terminal pedir a senha da chave. Rode uma vez:

```
[ -f ~/.bashrc ] && echo -e '\nexport GPG_TTY=$(tty)' >> ~/.bashrc
source ~/.bashrc
```

Quem usa **zsh** coloca a linha `export GPG_TTY=$(tty)` no `~/.zshrc`.

9.3.9 Passo 8 — Testar

```
git commit --allow-empty -m "Testando a assinatura"
git log --show-signature -1
```

```
commit e55fbf5be5b86eb89737adea712d8c605b5f9529
gpg: Assinatura feita seg 28 set 2026 14:51:16 -03
gpg:                usando RSA chave 0
      ↪ D9FFCDB3C4EBDC3E7225C8CB4BEA86F1F6C7956
gpg: Assinatura correta de "Maria Exemplo <maria@exemplo.com>" [final]
Author: Maria Exemplo <maria@exemplo.com>
```

A linha “Assinatura correta” (em inglês, “Good signature”) confirma que funcionou. Se você não ligou o `commit.gpgsign`, assine um commit específico com `git commit -S -m "..."`.

9.4 Assinando com SSH

9.4.1 Requisitos

- **Git 2.34 ou mais novo** (`git --version`);
- **OpenSSH 8.1 ou mais novo** (`ssh -V`). A versão 8.7 tem a assinatura quebrada: nela, atualize para a 8.8 ou mais nova;

9.4.2 Passo 1 — Ter uma chave SSH

Se você fez o login por SSH no capítulo 4, já tem uma: normalmente `~/.ssh/id_ed25519.pub`. Se não tiver, crie:

```
ssh-keygen -t ed25519 -C "maria@exemplo.com"
```

Aperte para aceitar o local padrão e escolha uma senha. O arquivo terminado em `.pub` é a chave **pública**. O outro, sem extensão, é a **privada**, e nunca sai do seu computador.

9.4.3 Passo 2 — Dizer ao Git para assinar com SSH

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true
git config --global tag.gpgSign true
```

9.4.4 Passo 3 — Cadastrar como chave de assinatura no GitHub

1. Foto → **Settings** → **SSH and GPG keys** → **New SSH key**.
2. Em **Title**, dê um nome. Em **Key type**, escolha **Signing Key**.

3. Em **Key**, cole o conteúdo do arquivo `.pub` (`cat ~/.ssh/id_ed25519.pub`) e clique em **Add SSH key**.

No GitHub, a mesma chave entra duas vezes

Uma chave cadastrada como *Authentication Key* serve para `git push`, mas **não** para verificar assinaturas. Se quiser usar a mesma chave para as duas coisas, cadastre-a duas vezes: uma com cada tipo.

Pelo terminal, com o GitHub CLI, dá no mesmo. A opção `--type` existe nas versões do `gh` lançadas a partir de 2023. Se a sua recusar a opção, use a página web.

```
gh ssh-key add ~/.ssh/id_ed25519.pub --type signing --title "Notebook
↳ pessoal"
```

9.4.5 Passo 4 — Permitir a conferência no seu computador

O GitHub confere a assinatura sozinho. Mas, para o *seu* Git conferir, ele precisa de uma lista de chaves confiáveis. Sem ela, aparece:

```
error: gpg.ssh.allowedSignersFile needs to be configured and exist for
↳ ssh signature verification
```

Crie a lista uma vez:

```
touch ~/.ssh/allowed_signers
git config --global gpg.ssh.allowedSignersFile ~/.ssh/allowed_signers
echo "$(git config --get user.email) namespaces=\"git\" $(cat ~/.ssh/
↳ id_ed25519.pub)" >> ~/.ssh/allowed_signers
```

Cada linha do arquivo associa um e-mail a uma chave. O `namespaces="git"` limita a chave a assinaturas do Git.

9.4.6 Passo 5 — Testar

```
git commit --allow-empty -m "Testando a assinatura SSH"
git log --show-signature -1
```

```
commit 86613eb51dee51c80a61f2cf8f1532df427e60c5
Good "git" signature for maria@exemplo.com with ED25519 key SHA256:Z/
↳ dW7m0FmUlo/Po1CS0bYXo03bAPY2bTohQtnJY0PGc
Author: Maria Exemplo <maria@exemplo.com>
```

9.5 Conferindo no GitHub

Envie um commit assinado com `git push` e abra a lista de commits:

- Na aba **Commits** do repositório, o selo **Verified** aparece ao lado do commit. Clique nele para ver qual chave assinou.

No terminal, uma letra resume o estado de cada commit:

```
git log --format='%h %G? %s'
```

G assinatura válida

N sem assinatura

B assinatura inválida

E não foi possível conferir (por exemplo, falta a chave pública ou o `allowed_signers`)

Vigilant mode no GitHub

Em **Settings** → **SSH and GPG keys**, a opção **Vigilant mode** (“Flag unsigned commits as unverified”) faz o GitHub marcar como **Unverified** todo commit seu que não esteja assinado, inclusive os que alguém fizer se passando por você. Ligue só depois que todos os seus computadores estiverem assinando.

9.6 Cuidados com a chave

- **Guarde a chave privada e a senha.** Se perder a chave, é só criar outra e cadastrar, mas perde-se a continuidade.
- **Chave vazou? Remova-a da conta e cadastre outra.** No GitHub, a verificação é registrada no momento do push: os commits antigos **continuam Verified** mesmo que a chave seja removida, expire ou seja revogada.
- **Commits feitos pela interface web** já saem assinados pela própria plataforma.
- No GitHub, um Pull Request integrado com **Rebase and merge** reescreve os commits, e eles perdem a sua assinatura.
- **Não é preciso publicar a chave em servidor de chaves (keyserver).** Alguns tutoriais pedem isso, mas o GitHub só usa a chave cadastrada na conta.

9.7 Problemas comuns

Mensagem ou sintoma	O que fazer
error: gpg failed to sign the data	quase sempre é o GPG_TTY (passo 7 do GPG). Confira também se o <code>user.signingkey</code> bate com <code>gpg --list-secret-keys</code>
gpg: signing failed: Inappropriate ioctl for device	falta o <code>export GPG_TTY=\$(tty)</code>
No secret key / secret key not available	o ID configurado não existe neste chaveiro. No Windows, confira se o Git usa o mesmo GPG em que a chave foi criada
gpg.ssh.allowedSignersFile needs to be configured and exist for ssh signature verification	crie o <code>allowed_signers</code> (passo 4 do SSH). A assinatura em si está certa
either user.signingkey or gpg.ssh.defaultKeyCommand needs to be configured	falta o <code>git config --global user.signingkey</code>
Selo Unverified no site	o e-mail do commit não está verificado na conta, a chave não foi cadastrada, ou a chave SSH foi cadastrada só como <i>Authentication Key</i>

Fontes oficiais

Os passos deste capítulo seguem a documentação do GitHub (GitHub 2026). Os nomes dos botões podem mudar com novas versões do site.

10 Exercícios práticos

Faça os exercícios na ordem. Todos os comandos são digitados no **Git Bash** (Windows) ou no **Terminal** (Linux e macOS).

Os materiais dos exercícios (`jogo-da-memoria.zip`, `main.py` e `modelo-actions.zip`) podem ser baixados pelos botões do topo da página <<https://andradasdev.github.io/github/>> ou na pasta `materials/` do repositório `ronidomingues/github-training`. Os comandos abaixo supõem que eles foram salvos na pasta `Downloads`.

Pasta de treino

Crie uma pasta só para os exercícios, por exemplo `~/treino`, e entre nela com `cd ~/treino` antes de começar.

10.1 Exercício 1 — Conferir as ferramentas

```
git --version
gh --version
gh auth status
```

Como saber se deu certo: as duas primeiras linhas mostram números de versão, e a terceira diz `Logged in to github.com account <seu-usuario>`. Se não estiver logado, volte ao capítulo 4.

10.2 Exercício 2 — O primeiro repositório

1. Crie o repositório no GitHub e já traga a cópia para o computador:

```
gh repo create meu-primeiro-repo --public --description "Meu
↳ primeiro repositório" --clone
cd meu-primeiro-repo
```

O Git avisa `You appear to have cloned an empty repository`. É normal: o repositório ainda não tem nenhum arquivo.

2. Crie o `README.md`, a “capa” do projeto no GitHub, escrita em Markdown:

```
cat > README.md << 'EOF'
# Meu Primeiro Projeto

Este é meu primeiro repositório no GitHub!
```

```
## O que este projeto faz?
```

```
- Aprender Git e GitHub  
- Praticar comandos  
EOF
```

3. Registre e envie:

```
git add README.md  
git commit -m "docs: cria o README"  
git push -u origin main
```

O que o -u faz

O -u liga a sua branch `main` à branch `main` do GitHub (origin). Isso só precisa ser feito no primeiro push: depois, basta `git push` e `git pull`.

Como saber se deu certo: a página `github.com/<seu-usuario>/meu-primeiro-repo` mostra o README formatado.

10.3 Exercício 3 — Uma licença criada pelo site

Este exercício mostra o caminho inverso: uma mudança feita no GitHub que precisa chegar ao seu computador.

1. Na página do repositório, clique em **Add file** → **Create new file**.
2. Digite `LICENSE` como nome. Aparece o botão **Choose a license template**: escolha **MIT License**, confira seu nome e clique em **Review and submit**.
3. Clique em **Commit changes**.
4. No terminal, traga a novidade:

```
git pull  
ls
```

Como saber se deu certo: o `ls` mostra o arquivo `LICENSE`, e o `git log --oneline` tem dois commits.

10.4 Exercício 4 — Resolver um conflito

1. **No site:** abra o `README.md` do `meu-primeiro-repo`, clique no lápis (**Edit**), troque a primeira linha por `# Meu Primeiro Projeto` (editado no site) e clique em **Commit changes**.

2. **No computador, sem dar pull:** abra o README.md num editor, troque a *mesma* primeira linha por # Meu Primeiro Projeto (editado no computador), salve e registre:

```
git commit -am "docs: edita o título no computador"
git push
```

O push é recusado, porque o GitHub tem um commit que você não tem:

```
! [rejected]          main -> main (fetch first)
error: failed to push some refs to 'https://github.com/...'
```

3. Traga a alteração do site:

```
git pull
```

```
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

4. Abra o README.md. O Git colocou as duas versões dentro dele:

```
<<<<<<< HEAD
# Meu Primeiro Projeto (editado no computador)
=====
# Meu Primeiro Projeto (editado no site)
>>>>>>> 15261e4...
```

Entre <<<<<<< e ===== está a sua versão; entre ===== e >>>>>>>, a que veio do GitHub. Deixe só a linha que deve ficar (ou escreva uma terceira), apague os três marcadores e salve.

5. Registre a resolução e envie:

```
git add README.md
git commit -m "docs: resolve o conflito do título"
git push
```

Como saber se deu certo: git log --oneline --graph mostra as duas linhas de trabalho se juntando no commit de resolução.

Atenção

Se o git pull terminar em Need to specify how to reconcile divergent branches, rode git config --global pull.rebase false e repita o pull.

10.5 Exercício 5 — Um site no GitHub Pages

1. Crie o repositório do site:

```
cd ~/treino
gh repo create meu-site --public --description "Meu primeiro site"
  ↳ --clone
cd meu-site
```

2. Descompacte o jogo-da-memoria.zip dentro de Downloads (no Windows: botão direito → **Extrair tudo**). Ele cria a pasta jogo-da-memoria/. Copie o **conteúdo** dela, e não a pasta, para dentro do meu-site:

```
cp -r ~/Downloads/jogo-da-memoria/. .
ls
```

O ls precisa mostrar o index.html aqui, na raiz.

3. Registre e envie:

```
git add .
git commit -m "feat: adiciona o jogo da memória"
git push -u origin main
```

4. No GitHub: **Settings** → **Pages**. Em **Source**, escolha **Deploy from a branch**; em **Branch**, main e /(root); clique em **Save**.

Como saber se deu certo: depois de um ou dois minutos, o jogo abre em <https://<seu-usuario>.github.io/meu-site/>.

10.6 Exercício 6 — Automação com GitHub Actions

1. Crie o repositório e copie o main.py para dentro dele:

```
cd ~/treino
gh repo create meu-script-python --public --description "Script
  ↳ Python com GitHub Actions" --clone
cd meu-script-python
cp ~/Downloads/main.py .
```

2. Crie a pasta dos workflows e o arquivo:

```
mkdir -p .github/workflows
```

Abra um arquivo novo .github/workflows/python.yml no editor e cole o Exemplo 2 do capítulo 8.

3. Envie:

```
git add .
git commit -m "ci: roda o script Python a cada push"
git push -u origin main
```

4. Acompanhe na aba **Actions** do repositório, ou com `gh run watch`. Quando terminar, traga o arquivo gerado:

```
git pull
cat resultado.txt
```

Como saber se deu certo: o `resultado.txt` mostra o horário da execução, e o último commit do repositório é do `github-actions[bot]`.

Outro modelo

O `modelo-actions.zip`, em `materials/`, traz o mesmo fluxo para um programa em Fortran: compila, executa e grava o resultado.

10.7 Exercício 7 — Fork, Pull Request e lista de presença

A lista de presença deste guia (Anexo A) é feita por Pull Request. É o fluxo real de contribuir com o projeto de outra pessoa.

1. Crie o seu fork e clone:

```
cd ~/treino
gh repo fork ronidomingues/github-training --clone
cd github-training
```

2. Crie uma branch para a sua contribuição:

```
git switch -c presenca/seu-nome
```

3. Crie um arquivo com o seu nome na pasta `presences/`. Use só letras, números, espaços, pontos e hífens no nome. Se quiser, escreva uma avaliação da capacitação dentro dele.

```
echo "Gostei muito da parte de conflitos!" > "presences/Seu Nome.txt"
↵ "
```

4. Registre e envie para o seu fork:

```
git add presences
git commit -m "chore(presences): registra a presença de Seu Nome"
git push -u origin presenca/seu-nome
```

5. Abra o Pull Request. No GitHub, na página do seu fork, clique em **Compare & pull request**, confira que a base é `ronidomingues/github-training` e clique em **Create pull request**. Pelo terminal:

```
gh pr create --repo ronidomingues/github-training --fill
```

Como saber se deu certo: o PR aparece na aba **Pull requests** do repositório original. Depois que ele for aceito, o seu nome entra na lista de presença da próxima versão do guia.

10.8 Exercício 8 — Mensagens de commit

Reescreva cada mensagem no formato Conventional Commits (capítulo 7):

1. “arrumei o erro que dava quando a senha tava vazia”
2. “coloquei explicação de como instalar no readme”
3. “agora o relatório sai em pdf também”
4. “atualizei o workflow pra rodar os testes”

Respostas possíveis

```
fix(login): aceita senha vazia sem erro · docs(readme): explica a
instalação · feat(relatorio): exporta em PDF · ci: roda os testes no
workflow
```

10.9 Exercício 9 — Um commit assinado

Siga o capítulo 9 com GPG ou SSH e faça um commit no `meu-primeiro-repo`:

```
git commit --allow-empty -m "chore: testa a assinatura"
git push
git log --format='%h %G? %s' -1
```

Como saber se deu certo: o terminal mostra a letra **G**, e a aba **Commits** do GitHub mostra o selo **Verified**.

10.10 Checklist

- ☐ Git e GitHub CLI instalados e logados
- ☐ Repositório criado, com README e LICENSE

- ☐ Um conflito resolvido
- ☐ Site publicado no GitHub Pages
- ☐ Workflow rodando no GitHub Actions
- ☐ Pull Request de presença aberto
- ☐ Commit assinado com selo **Verified**

11 Conclusão e próximos passos

Em duas horas, o caminho foi do zero ao fluxo completo de um projeto no GitHub: conta protegida e benefícios de estudante, Git e GitHub CLI instalados, commits bem escritos, branches, conflitos resolvidos, contribuição por fork e Pull Request, um site no ar pelo GitHub Pages, uma automação no GitHub Actions, arquivos grandes com o Git LFS e commits assinados.

Mais importante que decorar comandos é entender o modelo por trás deles: o histórico é uma sequência de commits, uma branch é um ponteiro, e um Pull Request é uma conversa antes de integrar. Com isso, os comandos novos passam a fazer sentido sozinhos.

11.1 Para continuar estudando

- **Pro Git**, de Scott Chacon e Ben Straub: o livro de referência do Git, gratuito e com tradução para o português, <<https://git-scm.com/book/pt-br/v2>>.
- **GitHub Skills**: cursos interativos dentro do próprio GitHub, <<https://skills.github.com>>.
- **Learn Git Branching**: exercícios visuais de branch e merge, em português, <https://learngitbranching.js.org/?locale=pt_BR>.
- **Documentação do GitHub**: <<https://docs.github.com/pt>>.

O próximo passo

Escolha um projeto seu, mesmo pequeno, e coloque no GitHub com README, licença e um workflow. Um repositório bem cuidado vale mais no portfólio do que muitos repositórios vazios.

Referências

Adorno 2022 ADORNO, R. *Padrões de Commits (Commit Patterns)*. 2022. <<https://dev.to/renatoadorno/padroes-de-commits-commit-patterns-41co>>. Acesso em: 28 set. 2026.

Conventional Commits Conventional Commits. *Conventional Commits 1.0.0*. <<https://www.conventionalcommits.org/pt-br/v1.0.0/>>. Acesso em: 28 set. 2026.

Git GIT. *Git documentation*. <<https://git-scm.com/docs>>. Acesso em: 28 set. 2026.

Git 2026 GIT. *Git for Windows: download*. 2026. <<https://git-scm.com/downloads/win>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *About large files on GitHub*. 2026. <<https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *About mandatory two-factor authentication*. 2026. <<https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/about-mandatory-two-factor-authentication>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *About storage and bandwidth usage*. 2026. <<https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-storage-and-bandwidth-usage>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *Apply to GitHub Education as a student*. 2026. <<https://docs.github.com/en/education/about-github-education/github-education-for-students/apply-to-github-education-as-a-student>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *Getting free access to Copilot as a student*. 2026. <<https://docs.github.com/en/copilot/how-tos/manage-your-account/free-access-with-copilot-student>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *GitHub Actions documentation*. 2026. <<https://docs.github.com/en/actions>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *GitHub CLI manual*. 2026. <<https://cli.github.com/manual/>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *GitHub Pages documentation*. 2026. <<https://docs.github.com/en/pages>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *Installing gh on Linux and BSD*. 2026. <https://github.com/cli/cli/blob/trunk/docs/install_linux.md>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *Managing commit signature verification*. 2026. <<https://docs.github.com/en/authentication/managing-commit-signature-verification>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *Plans for GitHub Copilot*. 2026. <<https://docs.github.com/en/copilot/get-started/plans>>. Acesso em: 28 set. 2026.

GitHub 2026 GITHUB. *Skipping workflow runs*. 2026. <<https://docs.github.com/en/actions/managing-workflow-runs-and-deployments/managing-workflow-runs/skipping-workflow-runs>>. Acesso em: 28 set. 2026.

Microsoft 2026 MICROSOFT. *Gerenciador de Pacotes do Windows (winget)*. 2026. <<https://learn.microsoft.com/pt-br/windows/package-manager/winget/>>. Acesso em: 28 set. 2026.

Silva SILVA, I. *Padrões de Commits*. <<https://github.com/iuricode/padroes-de-commits>>. Repositório no GitHub. Acesso em: 28 set. 2026.

Apêndices

APÊNDICE A – Comandos do Git

Referência dos principais comandos do Git, organizados por categoria. A documentação completa está em <<https://git-scm.com/docs>>.

Tabela 9 – Comandos do Git

Comando	Categoria	Explicação Detalhada
git init	Configuração/Setup	Transforma o diretório atual em um repositório Git, criando o diretório .git. Pode ser executado com segurança em um diretório existente sem sobrescrever configurações.
git config	Configuração/Setup	Lê ou define variáveis de configuração em nível de sistema, global ou local. Essencial para definir a identidade (user.name, user.email) do autor do commit.
git clone [url]	Configuração/Setup	Cria uma cópia local de um repositório remoto. Configura automaticamente a referência 'origin' e faz o checkout da branch principal.
git add [file]	Snapshotting Básico	Merge alterações de um arquivo do Working Tree para o Index (Staging Area), preparando-o para o próximo commit.
git status	Snapshotting Básico	Exibe o estado da Working Tree e do Index, listando arquivos modificados, staged ou não rastreados.
git diff	Snapshotting Básico	Mostra as diferenças entre o Working Tree e o Index (alterações não staged).
git diff - staged	Snapshotting Básico	Mostra as diferenças entre o Index (Staging Area) e o último commit (HEAD).
Continua na próxima página		

Continuação da Tabela: Comandos do Git

Comando	Categoria	Explicação Detalhada
git commit -m "[msg]"	Snapshotting Básico	Salva o conteúdo atualmente no Index como um novo snapshot permanente (commit) na história.
git commit --amend	Manipulação Histórico	Altera o commit anterior, seja modificando sua mensagem ou adicionando/removendo arquivos. Isso reescreve o histórico, gerando um novo SHA.
git rm [file]	Gerenciamento Arquivo	Remove um arquivo do Working Tree e do Index. O uso de --cached remove apenas do Index, mantendo o arquivo local.
git mv [old][new]	Gerenciamento Arquivo	Move ou renomeia um arquivo de forma rastreada pelo Git.
git clean	Gerenciamento Arquivo	Remove arquivos não rastreados (untracked files) do Working Tree.
git reset --soft [hash]	Manipulação Histórico	Move o ponteiro HEAD para o commit, mas mantém o Index e o Working Tree intactos (alterações permanecem staged).
git reset --mixed [hash]	Manipulação Histórico	(Padrão) Move o HEAD para o commit e reseta o Index (desencena arquivos), preservando o Working Tree.
git reset --hard [hash]	Manipulação Histórico	Move o HEAD e reseta o Index e o Working Tree, descartando todas as mudanças locais desde o hash. Altamente destrutivo.
git branch	Branching/Navegação	Gerenciamento de branches: lista, cria ou deleta branches locais.
git checkout	Branching/Navegação	Comando legado multi-uso. Alterna entre branches ou restaura arquivos antigos/commits, podendo resultar em 'detached HEAD'.
Continua na próxima página		

Continuação da Tabela: Comandos do Git

Comando	Categoria	Explicação Detalhada
git switch	Branching/Navegação	Comando moderno focado em alternar branches. Atualiza a Working Tree e o Index. Utilizado para criar novas branches de forma segura.
git merge [branch]	Integração/Merge	Integra alterações de uma branch na atual, criando um 'merge commit' se houver divergência. Operação não-destrutiva.
git rebase [base]	Integração/Rebase	Move ou reaplica commits para uma nova base, reescrevendo o histórico para mantê-lo linear. Ideal para branches locais e não publicadas.
git rebase -i [base]	Integração/Rebase	Modo interativo do rebase, permitindo squash (combinação), edição ou reordenação de commits.
git cherry-pick [hash]	Integração/Portabilidade	Aplica as alterações introduzidas por um único commit específico na branch atual, criando um novo commit equivalente.
git revert [hash]	Manipulação Histórico	Cria um novo commit que desfaz as alterações introduzidas por um commit anterior. Usado para desfazer mudanças em histórico compartilhado de forma segura.
git fetch	Sincronização Remota	Baixa dados (objetos e refs) de um repositório remoto para o repositório local, sem alterar o Working Tree ou Index (operação segura).
git pull	Sincronização Remota	Equivalente a git fetch seguido por uma integração (default: merge). Pode alterar o estado local e causar conflitos imediatamente (operação menos segura).
Continua na próxima página		

Continuação da Tabela: Comandos do Git

Comando	Categoria	Explicação Detalhada
git push [re- mote][branch]	Sincronização Remota	Carrega commits locais para um repositório remoto. Exige uma operação fast-forward, a menos que <code>--force</code> seja utilizado.
git push -tags	Sincronização Remota	Envia tags locais para o repositório remoto.
git remote	Sincronização Remota	Gerencia os repositórios remotos rastreados (e.g., listar, adicionar, remover).
git log	Auditoria/Inspeção	Exibe o histórico de commits.
git shortlog	Auditoria/Inspeção	Fornece um resumo conciso do git log, agrupando commits por autor.
git show	Auditoria/Inspeção	Exibe informações detalhadas sobre um objeto Git (commit, tag, etc.).
git reflog	Auditoria/Recuperação	Registra as atualizações locais no HEAD e em outras referências, agindo como uma rede de segurança para recuperar commits perdidos após resets ou rebase.
git tag	Marcação/Utilitários	Cria, lista, deleta ou verifica objetos de tag, usados para marcar pontos estáticos (releases) no histórico.
git tag -a [name]	Marcação/Utilitários	Cria uma tag anotada (com metadados e mensagem), preferida para releases públicas.
git stash	Utilitários de Contexto	Salva temporariamente o Working Directory e o Index (alterações ainda sem commit) para permitir a troca de contexto.
git stash pop	Utilitários de Contexto	Aplica o último stash salvo e o remove da lista de stashes.
git stash apply	Utilitários de Contexto	Aplica o último stash salvo, mas o mantém na lista.
Continua na próxima página		

Continuação da Tabela: Comandos do Git

Comando	Categoria	Explicação Detalhada
git submodule	Utilitários Avançados	Inicializa, atualiza ou inspeciona submódulos (repositórios aninhados).
git worktree	Utilitários Avançados	Gerencia múltiplas Working Trees (checkouts) do mesmo repositório, permitindo trabalhar em várias branches simultaneamente.
gitk	Utilitários Avançados	O navegador de repositório Git (ferramenta GUI).
scalar	Utilitários Avançados	Ferramenta projetada para gerenciar repositórios Git de grande escala (Large Git Repositories).
git sparse-checkout	Utilitários Avançados	Reduz a Working Tree para um subconjunto de arquivos rastreados, otimizando o desempenho em repositórios massivos.

APÊNDICE B – Comandos do GitHub CLI

Referência dos principais comandos do GitHub CLI (gh), organizados por grupo. A lista completa, sempre atualizada, está no manual oficial (GitHub 2026).

Tabela 10 – Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh alias	set	Cria um alias para um comando gh, permitindo atalhos personalizados para comandos frequentes	gh alias set prc "pr create"
gh alias	list	Lista todos os aliases configurados no GitHub CLI	gh alias list
gh alias	delete	Remove um alias previamente configurado	gh alias delete prc
gh auth	login	Autentica o usuário no GitHub via navegador web ou token	gh auth login
gh auth	logout	Remove a autenticação do usuário atual	gh auth logout
gh auth	status	Exibe o status de autenticação atual e usuário conectado	gh auth status
gh auth	refresh	Renova a autenticação para um host específico	gh auth refresh --hostname github.com
gh auth	token	Exibe o token de autenticação atual	gh auth token
Continua na próxima página			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh browse	-	Abre o repositório atual no navegador web	gh browse
gh browse	--branch	Abre uma branch específica no navegador	gh browse --branch feature-branch
gh browse	--commit	Abre um commit específico no navegador	gh browse --commit abc123
gh browse	<número>	Abre a issue ou o pull request de número informado no navegador	gh browse 42
gh browse	--settings	Abre as configurações do repositório no navegador	gh browse --settings
gh browse	--wiki	Abre a wiki do repositório no navegador	gh browse --wiki
gh codespace	code	Abre um codespace no Visual Studio Code	gh codespace code
gh codespace	cp	Copia arquivos entre o sistema local e um codespace	gh codespace cp local.txt remote:./
gh codespace	create	Cria um novo codespace	gh codespace create
gh codespace	delete	Remove um codespace específico	gh codespace delete -c my-codespace
gh codespace	jupyter	Abre um codespace no JupyterLab	gh codespace jupyter
gh codespace	list	Lista todos os codespaces disponíveis	gh codespace list

Continua na próxima página

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh codespace	logs	Exibe os logs de um codespace específico	gh codespace logs -c my-codespace
gh codespace	ports	Lista as portas encaminhadas de um codespace	gh codespace ports
gh codespace	ports forward	Encaminha uma porta do codespace para o local	gh codespace ports forward 3000:4000
gh codespace	ports visibility	Define a visibilidade de uma porta	gh codespace ports visibility 3000:public
gh codespace	ssh	Conecta-se a um codespace via SSH	gh codespace ssh
gh codespace	stop	Para um codespace em execução	gh codespace stop -c my-codespace
gh gist	create	Cria um novo gist a partir de arquivos ou entrada padrão	gh gist create script.py
gh gist	clone	Clona um gist específico para o sistema local	gh gist clone abc123
gh gist	delete	Remove um gist específico	gh gist delete abc123
gh gist	edit	Edita um gist existente	gh gist edit abc123
gh gist	list	Lista todos os gists do usuário	gh gist list
gh gist	view	Visualiza um gist específico no terminal	gh gist view abc123
gh issue	create	Cria uma nova issue no repositório	gh issue create --title "Bug" --body "Descrição"
<i>Continua na próxima página</i>			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh issue	list	Lista issues do repositório com filtros opcionais	gh issue list --state open
gh issue	status	Mostra o status das issues relevantes para o usuário	gh issue status
gh issue	close	Fecha uma issue específica	gh issue close 42
gh issue	comment	Adiciona um comentário a uma issue	gh issue comment 42 --body "Comentário"
gh issue	delete	Remove uma issue específica	gh issue delete 42
gh issue	edit	Edita uma issue existente	gh issue edit 42 --title "Novo título"
gh issue	lock	Trava os comentários de uma issue	gh issue lock 42
gh issue	reopen	Reabre uma issue fechada	gh issue reopen 42
gh issue	transfer	Transfere uma issue para outro repositório	gh issue transfer 42 owner/repo
gh issue	view	Exibe detalhes de uma issue específica	gh issue view 42
gh project	copy	Copia um projeto para outro dono (usuário ou organização)	gh project copy 1 --source-owner @me --target-owner novaorg --title "Cópia" --drafts
gh project	create	Cria um novo projeto	gh project create --owner @me --title "Meu Projeto"
gh project	delete	Remove um projeto específico	gh project delete 1 --owner @me
<i>Continua na próxima página</i>			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh project	edit	Edita as propriedades de um projeto	gh project edit 1 --owner @me --title "Novo Título"
gh project	field-create	Cria um campo personalizado no projeto	gh project field-create 1 --owner @me --name "Prioridade" --data-type TEXT
gh project	item-add	Adiciona uma issue ou um pull request ao projeto	gh project item-add 1 --owner @me --url < https://github.com/owner/repo/issues/1 >
gh project	list	Lista projetos disponíveis	gh project list --owner owner
gh project	view	Visualiza detalhes de um projeto específico	gh project view 1 --owner @me
gh pr	checks	Exibe os status checks de um pull request	gh pr checks 15
gh pr	close	Fecha um pull request específico	gh pr close 15
gh pr	comment	Adiciona um comentário a um pull request	gh pr comment 15 --body "Comentário"
gh pr	create	Cria um novo pull request	gh pr create --title "Feature" --body "Descrição"
gh pr	diff	Exibe as diferenças introduzidas pelo pull request	gh pr diff 15
gh pr	edit	Edita propriedades de um pull request	gh pr edit 15 --title "Novo Título"
gh pr	list	Lista pull requests do repositório	gh pr list --state open
<i>Continua na próxima página</i>			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh pr	merge	Mescla um pull request	gh pr merge 15 --squash
gh pr	ready	Marca um pull request como pronto para revisão	gh pr ready 15
gh pr	reopen	Reabre um pull request fechado	gh pr reopen 15
gh pr	review	Adiciona uma revisão a um pull request	gh pr review 15 --approve
gh pr	status	Mostra o status dos pull requests relevantes	gh pr status
gh pr	view	Exibe detalhes de um pull request específico	gh pr view 15
gh pr	checkout	Faz checkout da branch de um pull request	gh pr checkout 15
gh release	create	Cria um novo release	gh release create v1.0.0 --title "Versão 1.0.0"
gh release	delete	Remove um release específico	gh release delete v1.0.0
gh release	download	Baixa os assets de um release	gh release download v1.0.0
gh release	list	Lista todos os releases do repositório	gh release list
gh release	upload	Faz upload de assets para um release	gh release upload v1.0.0 arquivo.zip
gh release	view	Exibe detalhes de um release específico	gh release view v1.0.0
<i>Continua na próxima página</i>			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh release	edit	Edita propriedades de um release existente	gh release edit v1.0.0 --title "Novo Título"
gh repo	archive	Arquiva um repositório	gh repo archive owner/repo
gh repo	clone	Clona um repositório para o sistema local	gh repo clone owner/repo
gh repo	create	Cria um novo repositório	gh repo create meu-repo --public
gh repo	delete	Remove um repositório	gh repo delete owner/repo
gh repo	edit	Edita propriedades de um repositório	gh repo edit --description "Nova descrição"
gh repo	fork	Cria um fork de um repositório	gh repo fork owner/repo
gh repo	list	Lista repositórios do usuário ou organização	gh repo list --limit 10
gh repo	rename	Renomeia um repositório	gh repo rename novo-nome
gh repo	sync	Sincroniza um fork com seu repositório upstream	gh repo sync
gh repo	view	Exibe detalhes de um repositório	gh repo view owner/repo
gh repo	deploy-key	Gerencia chaves de deploy do repositório	gh repo deploy-key add chave.pub --title "Servidor"
gh run	cancel	Cancela uma execução de workflow	gh run cancel 123456789
gh run	delete	Remove execuções de workflow	gh run delete 123456789
<i>Continua na próxima página</i>			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh run	download	Baixa artifacts de uma execução	gh run download 123456789
gh run	list	Lista execuções de workflows	gh run list
gh run	rerun	Reexecuta um workflow falho	gh run rerun 123456789
gh run	view	Exibe detalhes de uma execução	gh run view 123456789
gh run	watch	Monitora uma execução em tempo real	gh run watch 123456789
gh search	code	Busca por código no GitHub	gh search code "função javascript"
gh search	commits	Busca por commits	gh search commits "fix bug" --author=user
gh search	issues	Busca por issues e pull requests	gh search issues "bug label:bug"
gh search	prs	Busca especificamente por pull requests	gh search prs "feature state:open"
gh search	repos	Busca por repositórios	gh search repos "topic:machine-learning"
gh search	users	Busca por usuários	gh search users "nome location:Brasil"
gh secret	list	Lista secrets disponíveis	gh secret list
gh secret	remove	Remove um secret específico	gh secret remove API_KEY
gh secret	set	Define ou atualiza um secret	gh secret set API_KEY --body "valor"
gh ssh-key	add	Adiciona uma chave SSH à conta	gh ssh-key add chave.pub --title "Laptop"
gh ssh-key	list	Lista chaves SSH da conta	gh ssh-key list
<i>Continua na próxima página</i>			

Continuação da Tabela: Comandos do GitHub CLI (gh)

Comando Base	Subcomando	Explicação Funcional Detalhada	Exemplo de Sintaxe Chave
gh ssh-key	delete	Remove uma chave SSH	gh ssh-key delete 123
gh workflow	disable	Desabilita um workflow	gh workflow disable "CI Tests"
gh workflow	enable	Habilita um workflow	gh workflow enable "CI Tests"
gh workflow	list	Lista workflows disponíveis	gh workflow list
gh workflow	run	Executa um workflow manualmente	gh workflow run "CI Tests"
gh workflow	view	Exibe detalhes de um workflow	gh workflow view "CI Tests"

APÊNDICE C – Glossário

Branch Linha de desenvolvimento dentro do histórico; tecnicamente, um ponteiro para um commit.

Clone Cópia completa de um repositório remoto no seu computador.

Commit Registro de um estado do projeto, com autor, data e mensagem.

Conflito Situação em que duas alterações na mesma região de um arquivo não podem ser combinadas automaticamente.

Fork Cópia de um repositório de outra pessoa na sua conta, para você propor mudanças por Pull Request.

GitHub Actions Plataforma de automação do GitHub, descrita em arquivos YAML.

GitHub Pages Hospedagem de sites estáticos a partir de um repositório.

.gitignore Arquivo que lista o que o Git não deve versionar.

Merge Integração de duas linhas de histórico.

Origin Nome padrão do repositório remoto principal.

Pull Request Pedido para integrar uma branch em outra, com revisão.

README Arquivo de apresentação do repositório, em Markdown.

Remoto Cópia do repositório em outro lugar, como o GitHub.

Repositório Pasta cujo histórico é controlado pelo Git.

Workflow Processo automatizado do GitHub Actions.

Anexos

ANEXO A – Lista de presença

Participantes que registraram a presença por Pull Request (Exercício 7, seção 10.7). A data é a do momento em que o Pull Request foi integrado ao repositório.

Nome	Presença registrada em
Ronivaldo D. Andrade	09/10/2025 às 12:04
Persona Presente	09/10/2025 às 14:35